Military Technical College

Chair of Computing Engineering and Artificial Intelligence



Training-Free Adaptive Control of Detector and Tracker Operating Points for Multi-Object Tracking

By

Ahmed Gouda Ismail
Military Technical College

Under supervision of

Assoc. Prof. Dr. Mohamed S. Mohamed
Military Technical College

Prof. Dr. Tarek Ahmed Mahmoud
Egypt University of Informatics

This thesis is submitted in partial fulfillment of the requirements for the degree of

Master of Science In Computer Engineering

Cairo 2026

Military Technical College

Chair of Computing Engineering and Artificial Intelligence

Training-Free Adaptive Control of Detector and Tracker Operating Points for Multi-Object Tracking

By

Ahmed Gouda Ismail
Military Technical College

Under supervision of

Assoc. Prof. Dr. Mohamed S. Mohamed
Military Technical College

Prof. Dr. Tarek Ahmed Mahmoud
Egypt University of Informatics

This thesis is submitted in partial fulfillment of the requirements for the degree of

Master of Science In Computer Engineering

Date of Discussion Choose an item. Choose an item. **2026**

Statement of committee approval

Rank and Degree	Name	Signature
Discussion Committee		
PGS-Reviewer		
Head of PGS		

Cairo 2026

Abstract

Multi-object tracking (MOT) finds every object of interest in a video and keeps one identity for each object over time. In tracking by detection, a detector returns scored boxes in every frame and a tracker links them into tracks. Both run with fixed settings, the operating point: the detector uses one confidence threshold, one non-maximum suppression (NMS) threshold, and one input resolution, and the tracker keeps score thresholds that were tuned for one detector. Yet scenes change within a video, above all in unmanned aerial vehicle (UAV) footage, and detectors are replaced while trackers are not retuned. This thesis proposes two training-free and causal controllers that adapt these operating points online, without labels and without changing the detector or the tracker.

The first controller, Adaptive Control for Multi-Object Tracking (AC-MOT), sets the detector operating point in UAV video. Every ten frames it forms a scene complexity index from cheap image and track cues and selects one of three operating points; its values were selected by cross-validation on calibration sequences and frozen. On seven VisDrone2019-MOT sequences it raised the higher order tracking accuracy (HOTA) by 2.937 points with ByteTrack and 1.325 points with OATrack, and the multiple object tracking accuracy (MOTA) by 17.995 and 6.318 points. On UAVDT, without retuning, it raised MOTA by 17.601 and 3.267 points, at about 4.2 ms per frame.

The second controller, a self-calibrating control layer (SCL), sits between an unchanged detector and an unchanged tracker. It splits the score logits of recent frames twice with Otsu's method, keeps the tracker's scores and thresholds when the scores are clean, and otherwise chooses the candidates, rescores them, and sets the thresholds. One frozen configuration was tested on seven published trackers, four detection sources, and three datasets. It restored two trackers from 0 to about 66 HOTA under halved scores, improved both single-stage trackers on every MOT17 sequence, raised MOTA by up to 24.70 points with a noisy detector on held-out aerial data, reached higher HOTA than simple adaptive thresholds and the published ACT method, and takes 2.95 ms per frame on an ordinary processor.

The results show that the detector operating point and the score scale that the tracker receives can both be controlled online, cheaply, and without training.

Keywords: multi-object tracking, tracking by detection, detector operating point, confidence calibration, training-free control, unmanned aerial vehicles, Otsu thresholding.

Acknowledgments

Special acknowledgment is due to those who have contributed to this work directly and indirectly. I would like to express my deep gratitude to my supervisors, Assoc. Prof. Dr. Mohamed S. Mohamed and Prof. Dr. Tarek Ahmed Mahmoud, for their guidance, their patience, and their careful reading of every result throughout this research. Their advice shaped both the methods and the way they were evaluated.

I am grateful to the Computer Engineering and Artificial Intelligence Department of the Military Technical College for its support during my studies, and to the authors of the published detectors and trackers used in this thesis and the maintainers of the VisDrone, UAVDT, MOT17, and KITTI benchmarks and of the TrackEval library, whose released code and data made a reproducible evaluation possible.

Finally, I would like to thank my family for their patience, encouragement, and support.

Ahmed
Gouda Ismail

Cairo 2026

Contents

Abstract ..... **I**
Acknowledgments ..... **II**
List of Figures ..... **VI**
List of Tables ..... **VII**
List of Abbreviations ..... **IX**
Chapter 1 Introduction **1**
1.1 Overview 1
1.2 Motivation and Challenges 2
1.3 Problem Statement 3
1.4 Proposed Solution 4
1.5 Thesis Organization 5
Chapter 2 Background and Literature Review **7**
2.1 Introduction 7
2.2 Multi-Object Tracking Overview 7
2.2.1 Evolution of Object Detection and Tracking 7
2.2.2 Applications and Challenges 8
2.2.3 Real-Time Constraints 8
2.2.4 Formal Definition of Tracking by Detection 8
2.2.5 Multi-Object Tracking Paradigms 9
2.3 Object Detectors 10
2.3.1 Two-Stage Detectors 10
2.3.2 Single-Stage Detectors 10
2.3.3 Transformer-Based Detectors 10
2.3.4 Detector Confidence and Calibration 11
2.4 Tracking-by-Detection Trackers 11
2.4.1 Motion-Based Trackers 12
2.4.2 Motion and Appearance Trackers 12
2.4.3 Recent Trackers Used in This Thesis 12
2.4.4 How Trackers Use Detector Scores 13
2.5 Datasets for Detection and Tracking 14
2.6 Literature Review 14
2.6.1 Adaptive Inference for Object Detection 14

2.6.2 Adaptive Thresholds in Multi-Object Tracking....15
2.6.3 Aerial Multi-Object Tracking....15
2.6.4 Optimization and Evaluation Tools....15
2.6.5 Summary of the Literature and Research Gap....16
Chapter 3 Proposed Training-Free Adaptive Control Approach....17
3.1 Introduction....17
3.2 Scene-Adaptive Detector Operating-Point Controller (AC-MOT)....17
3.2.1 Overview and Operating Point....17
3.2.2 Scene Cues....18
3.2.3 Scene Complexity Index and Scene Label....19
3.2.4 Three-State Controller....20
3.2.5 Action Map....23
3.2.6 Selection of Parameter Values....24
3.3 Development of the AC-MOT Controller....25
3.3.1 Quality-Oriented Profile....26
3.3.2 Identity-Aware Pareto Profile....27
3.3.3 Transfer and Cross-Pipeline Studies....28
3.3.4 Development of the Frozen Controller....29
3.4 Self-Calibrating Control Layer (SCL)....31
3.4.1 Overview....31
3.4.2 Inputs and Outputs....32
3.4.3 Self-Calibration from Detector Scores....33
3.4.4 Clean and Noisy Modes....34
3.4.5 Track Continuation and Camera Motion....35
3.4.6 Host Contract and Adapters....36
3.4.7 Parameter Values....36
3.5 Development of the SCL....38
3.5.1 Starting Point: Always-On Banding....38
3.5.2 Design Steps....39
3.5.3 Freezing and Testing....39
3.6 Summary....39
Chapter 4 Experimental Results and Performance Evaluation....41
4.1 Introduction....41
4.2 Setup Discussion....41

4.2.1 Datasets..... 41
4.2.2 Evaluation Metrics..... 43
4.2.3 Statistical Comparison..... 45
4.2.4 Implementation Environment and Toolkit..... 45
4.3 Results of the Scene-Adaptive Controller (AC-MOT)..... 46
4.3.1 Frozen VisDrone Evaluation..... 46
4.3.2 Controller Behavior..... 48
4.3.3 Frozen UAVDT Transfer..... 48
4.3.4 Runtime of AC-MOT..... 50
4.4 Results of the Self-Calibrating Control Layer (SCL)..... 50
4.4.1 MOT17 Evaluation Across Trackers..... 51
4.4.2 KITTI Evaluation with Other Detectors..... 52
4.4.3 Evaluation under Shifted Detector Scores..... 54
4.4.4 Frozen Evaluation on External Trackers..... 55
4.4.5 Frozen UAVDT Transfer..... 56
4.4.6 Runtime of SCL..... 56
4.5 Comparison with Existing Approaches..... 56
4.5.1 Comparison with Adaptive-Threshold Baselines..... 56
4.5.2 Comparison with ACT..... 57
4.5.3 Contribution of Each Part of SCL..... 57
4.5.4 Comparison of Properties..... 58
4.6 Discussion..... 58
Chapter 5 Conclusions and Future Work..... 60
5.1 Conclusion..... 60
5.2 Future Work..... 61
References..... 62
Publications..... 70

List of Figures

Fig. No. **Title** **Page No.**

Fig. (1.1) Tracking-by-detection pipeline and its operating points.....2
Fig. (1.2) Outline of the proposed approach.....5
Fig. (2.1) How a two-stage and a single-stage tracker use the detector score, with the published thresholds of ByteTrack (official setting) and OC-SORT.....13
Fig. (3.1) AC-MOT in a tracking-by-detection pipeline.....18
Fig. (3.2) SCL between a frozen detector and a frozen tracker (the host).....32
Fig. (3.3) What the layer sees: the logits of the candidates of the last 10 frames, just before its decision for frame 101, with the Otsu splits t1 (dashed line) and t2 (solid line).....34
Fig. (4.1) HOTA, MOTA, and IDF1 on the seven VisDrone sequences for both tracker hosts (values in Table (4.3)).....47
Fig. (4.2) HOTA, MOTA, and IDF1 of the frozen UAVDT transfer of AC-MOT for both tracker hosts (values in Table (4.5)).....50
Fig. (4.3) Difference in HOTA, host with SCL minus host alone, in points, for every tested cell, with its 95% paired-bootstrap interval.....51
Fig. (4.4) Tracking on KITTI sequence 0001 [3] with OC-SORT and YOLOv8n, at frames 92, 97, and 102 (rows): (a) the tracker alone, (b) with SCL.....54

List of Tables

Table No. **Title** **Page No.**

Table (2.1) Detector families discussed in this thesis..... 11
Table (2.2) Tracking-by-detection trackers discussed in this thesis and how they use detector scores. 13
Table (2.3) Comparison table of the literature review: methods closest to the proposed controllers.. 16
Table (3.1) Action map of AC-MOT..... 24
Table (3.2) Values of the AC-MOT controller and how each value is chosen.....25
Table (3.3) Quality-oriented profile versus its baseline (YOLOv8n with ByteTrack at default settings) on VisDrone test-dev, historical protocol..... 27
Table (3.4) Identity-aware profile versus its baseline (YOLOv8n with ByteTrack at default settings) on VisDrone test-dev, historical protocol..... 28
Table (3.5) Historical UAVDT transfer without retuning..... 28
Table (3.6) Cross-pipeline studies..... 29
Table (3.7) Frozen AC-MOT policy versus the modern baseline (YOLO11m at 1536 pixels and NMS 0.70, ByteTrack) on the four calibration folds: change in HOTA.....31
Table (3.8) Host contracts: the published settings of each tracker, as used by the layer: matching (association) threshold a, birth threshold b, low threshold ℓ, and match tolerance m0.....36
Table (3.9) Values of SCL and their source.....37
Table (3.10) Baseline comparison of the development stage on MOT17 val-half (published YOLOX-X detections)..... 38
Table (3.11) Design steps from AOB to SCL on development data (HOTA, %), and versions that were tested and rejected..... 39
Table (4.1) Datasets and splits used in the experiments and the role of each one.....42
Table (4.2) Detectors, trackers, data, and hardware used to evaluate the two controllers.....46
Table (4.3) Frozen VisDrone evaluation on seven VisDrone2019-MOT sequences (71,830 ground-truth boxes)..... 47
Table (4.4) Paired sequence-level bootstrap of AC-MOT: AC-MOT minus baseline, 5000 resamples, seed 42, on VisDrone (7 sequences) and UAVDT (20 sequences).....48
Table (4.5) Frozen UAVDT transfer of AC-MOT without retuning (20 sequences, 16,592 frames, one vehicle class)..... 49
Table (4.6) Baseline comparison of the development trackers on MOT17 val-half (published YOLOX-X detections)..... 52
Table (4.7) Baseline comparison under a detector change on KITTI tracking training (official HOTA, car and pedestrian averaged)..... 53
Table (4.8) Baseline comparison under shifted detector scores on MOT17 val-half.....54

Table (4.9) Baseline comparison on the predeclared external systems (MOT17 val-half, authors' released detections), each evaluated once with the frozen SCL.....55
Table (4.10) Baseline comparison on the held-out UAVDT test split (20 aerial sequences, not used during design), evaluated once with the frozen SCL.....56
Table (4.11) HOTA (%) of SCL and of two simple adaptive-threshold baselines on KITTI tracking training (official HOTA, car and pedestrian averaged).....57
Table (4.12) HOTA of ACT and of SCL on the same detections, and the differences SCL minus ACT in HOTA and IDF1, with 95% paired-bootstrap interval; bold intervals exclude zero.....57
Table (4.13) Adaptive methods closest to the proposed controllers.....58
Table (4.14) Summary of the two proposed controllers.....59

List of Abbreviations

Abbreviations

AC-MOT	Adaptive Control for Multi-Object Tracking
AOB	Always-on banding
AssA	Association accuracy
CI	Confidence interval
CNN	Convolutional neural network
COCO	Common Objects in Context
CPU	Central processing unit
CV	Cross-validation
DetA	Detection accuracy
DETR	Detection transformer
FN	False negatives
FP	False positives
FPS	Frames per second
GPU	Graphics processing unit
GT	Ground truth
HOTA	Higher order tracking accuracy
IDF1	Identity F1 score
IDFN	Identity false negatives
IDFP	Identity false positives
IDS	Identity switches
IDTP	Identity true positives
IoU	Intersection over union
MOT	Multi-object tracking
MOTA	Multiple object tracking accuracy
NMS	Non-maximum suppression
ReID	Re-identification
RT-DETR	Real-time detection transformer
SCI	Scene complexity index
SCL	Self-calibrating control layer
TP	True positives
TPE	Tree-structured Parzen estimator
UAV	Unmanned aerial vehicle
YOLO	You Only Look Once

Chapter 1
Introduction

1.1 Overview

Multi-object tracking (MOT) finds every object of interest in a video and gives each one an identity number (ID) that stays the same over time. It is a basic function of surveillance, traffic monitoring, driver assistance, and drone systems [1–3], where the results are usually needed with little delay. Unmanned aerial vehicles (UAVs) in particular are widely used for traffic monitoring, surveillance, and search, and many of these tasks need every object to be detected in every frame and to keep the same identity while it is visible [1, 2].

Most MOT systems follow the tracking-by-detection design [4–6]. In each frame, a detector returns candidate boxes, each with a class and a confidence score, and an online tracker links these boxes into tracks, one per object. The two parts are developed separately and are combined freely: any detector can feed any tracker. This modularity is the main strength of the design, and it is also the source of the problem studied in this thesis.

Both parts of the pipeline run with a small number of settings that decide which boxes are used (Fig. (1.1)). The detector removes boxes whose score is below its confidence threshold, removes overlapping boxes with non-maximum suppression (NMS), and resizes every frame to one input resolution. These three values form the *detector operating point*. The tracker, in turn, uses a box only if its score passes its own thresholds: one score to match a box to an existing track, one score to start a new track, and, in two-stage trackers such as ByteTrack, a lower score for a second matching step [6]. In almost every system these settings are fixed once, for one detector and one type of scene, and they stay the same for the whole video.

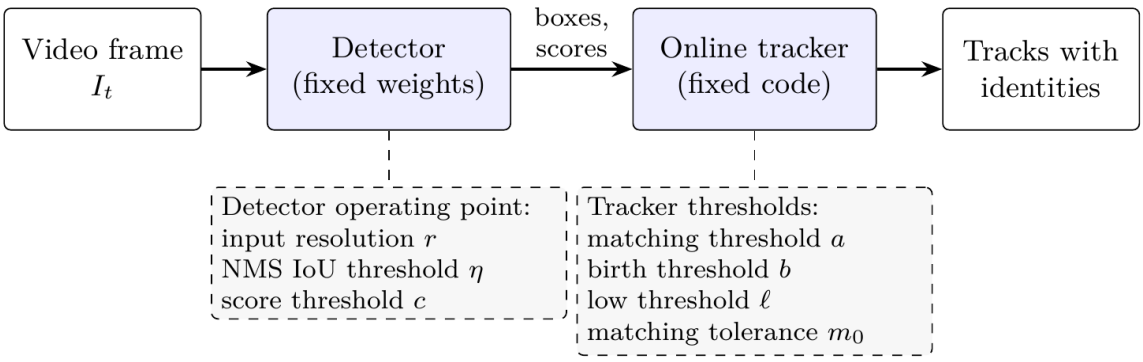


Fig. (1.1) Tracking-by-detection pipeline and its operating points.

The detector settings decide which boxes reach the tracker, and the tracker thresholds decide which of these boxes may match or start a track. In common practice all of these values are fixed for the whole video.

This thesis treats these settings as control variables of the tracking system rather than as fixed parameters. It proposes two training-free controllers that adapt them online, from information that is available while the video is processed, with the detector and the tracker left unchanged.

1.2 Motivation and Challenges

A fixed operating point does not fit a scene that keeps changing. Aerial video makes this especially clear: objects are small, their number changes quickly, the camera moves, and illumination and blur change within one video [1, 7, 8]. A setting that works in a sparse, clear scene can keep too much clutter in a dense scene, and a setting that works in a dense scene can lose small or weak objects in another part of the same video. A large input helps small objects, but it costs the same time on easy and hard frames. The choice therefore moves false detections, missed objects, identity switches, and runtime together.

A second problem appears when the detector changes. In real systems the detector is often replaced, retrained, or compressed, while the tracker keeps the thresholds that its authors set for another detector. Detector scores are not calibrated across models and conditions [9], so a new detector usually gives scores on a different scale, and nothing in the system notices. The effect can be large. ByteTrack in its official setting starts a track only from a box with a score of at least 0.7; if the scores of a published detector are halved, a car that was scored 0.9 now gets 0.45, ByteTrack never starts a track, and its higher order tracking accuracy (HOTA) [10] falls from 67.70 to 0 (Section 4.4.3).

The main challenges of adapting the operating point in a working system are:

- *No labels at run time.* A deployed system has no ground truth, so the adaptation must be decided from the video and from the system's own outputs.
- *Causality and delay.* Only the current and earlier frames may be used, and the extra computation must be small next to the detector, so that the system stays usable in real time.
- *Unchanged components.* Retraining the detector or rewriting the tracker for every new combination is costly and loses the modularity of tracking by detection; the controller should work from the outside.
- *Stability.* Each change of the operating point changes the boxes that reach the tracker, so the settings should follow lasting changes of the scene rather than single frames.
- *No harm when nothing is wrong.* A tracker that already fits its detector should keep its accuracy; an adaptation that always intervenes can lower it.

Existing methods address parts of this problem. Adaptive inference methods change the detector configuration, but they usually need an extra learned model and target detection rather than tracking [11–13]. Adaptive-threshold methods change thresholds inside one specific tracker [14–16], and score calibration methods need labelled data [9]. A simple, training-free way to adapt the operating point of a tracking pipeline online, with the detector and the tracker left unchanged, is missing.

1.3 Problem Statement

Let a video be a sequence of frames I_1, I_2, …, processed online by a frozen detector and a frozen tracker. The weights of the detector, and the code and association algorithm of the tracker, are not changed. At every frame t, a controller may choose only settings that the pipeline already exposes, and it may base its choice only on the current frame, on earlier frames, and on the outputs that the detector and the tracker have already produced. No ground truth, no future frame, and no training is allowed. The goal is to improve tracking accuracy, measured by HOTA, the multiple object tracking accuracy (MOTA), the identity F1 score (IDF1), and the number of identity switches (IDS), while adding little computation.

The thesis studies two forms of this problem:

1. *Scene-adaptive detector operating point.* In UAV video, choose the confidence threshold, the NMS threshold, and the input resolution of the detector from cheap cues of the scene, so that the boxes passed to the tracker fit the current scene.

2. *Self-calibration to the detector's score scale.* When the detector or its scores change, choose which candidates reach the tracker, the scores they carry, and the tracker's thresholds for the frame, from the scores of recent frames alone, so that a published tracker keeps working with a detector it was not tuned for, and is left as it is when it already fits.

1.4 Proposed Solution

The thesis proposes two rule-based controllers that share the same principles: no learned weights, no labels at run time, use of past information only, unchanged detector and tracker, values fixed before the final evaluation, and paired bootstrap statistics for every comparison. Figure (1.2) shows where they act in the pipeline.

The first controller, Adaptive Control for Multi-Object Tracking (AC-MOT), runs next to the detector. Every ten frames it measures cheap cues of the current image and of the boxes that the tracker reported in the previous frame, averages the recent crowd level into a scene complexity index (SCI), and selects one of three detector operating points through a three-state controller with recovery, hysteresis, and dwell rules. Its thresholds and actions were selected by cross-validation on separate calibration sequences and then frozen.

The second controller, a self-calibrating control layer (SCL), runs once per frame between the detector and the tracker. It splits the score logits of the last ten frames twice with Otsu's method [17] into background, uncertain, and confident groups, and decides from the share of confident candidates whether the score distribution is clean or noisy. On clean scores it keeps the tracker's scores and thresholds; on noisy scores it chooses the candidates, gives them new scores, and sets the thresholds. A continuation step lets a weak box that clearly continues a track reach single-stage trackers, which have no low-score step of their own. The layer serves different published trackers through four of their published settings, called the host contract.

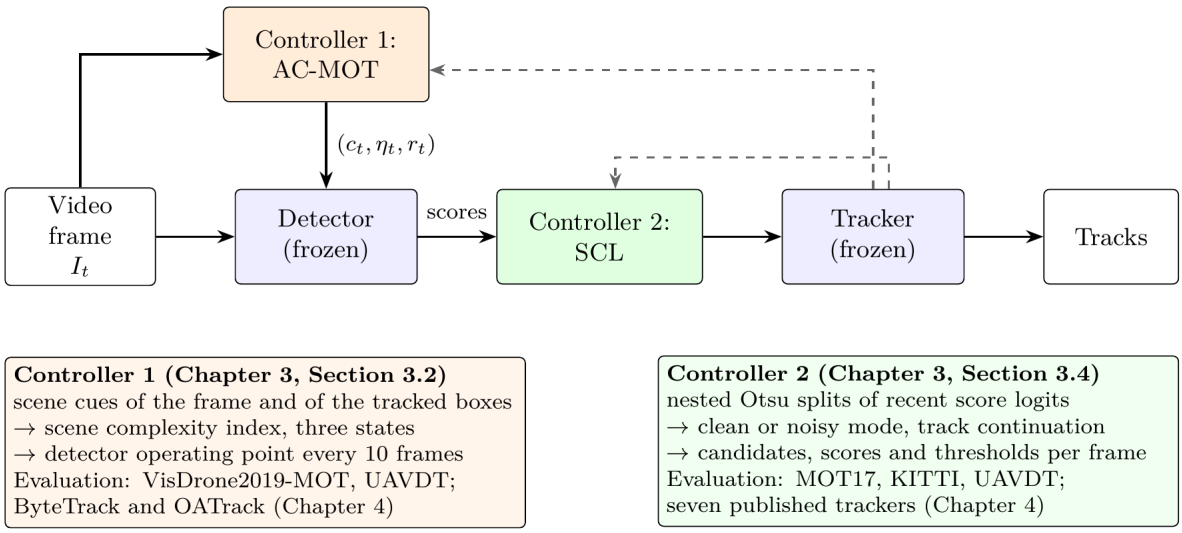


Fig. (1.2) Outline of the proposed approach.

Controller 1 (AC-MOT) sets the detector operating point from scene cues; Controller 2 (SCL) adapts the candidates, scores, and thresholds that the tracker receives from the detector's score distribution. Dashed arrows: tracked boxes of the previous frame.

The contribution of this research is summarized as follows:

- AC-MOT, a training-free and causal controller that treats the detector operating point as part of the MOT system and adapts it online, with the detector and the tracker unchanged, together with an optimization-guided development path whose final values are selected by cross-validation on calibration sequences.
- An evaluation of AC-MOT on VisDrone2019-MOT with two trackers, ByteTrack and OATrack, using eight MOT metrics and paired bootstrap intervals, and a transfer to UAVDT without retuning.
- SCL, a training-free and causal layer that keeps the tracker's scores and thresholds when the scores are clean and takes control only when they are noisy, with a host contract that lets one fixed configuration serve single-stage and two-stage trackers without changing them.
- An evaluation of one frozen SCL configuration on seven published trackers, four detection sources, and three datasets, including score shifts, predeclared external trackers, a held-out aerial test split, and comparisons with simple adaptive thresholds and the published ACT method.
- Runtime measurements showing that both controllers add only a few milliseconds per frame.

1.5 Thesis Organization

The rest of this thesis is organized as follows.

Chapter 2: Background and Literature Review

This chapter reviews object detection and multi-object tracking, the detectors and trackers used in this work, detector confidence and its calibration, and the related work on adaptive inference, adaptive thresholds, and aerial tracking. It ends with a comparison table of the closest methods and the gap that this thesis addresses.

Chapter 3: Proposed Training-Free Adaptive Control Approach

This chapter presents the two proposed controllers. It describes the scene cues, the scene complexity index, the three-state controller, and the action map of AC-MOT, and how its values were selected; then the score self-calibration, the clean and noisy modes, the track continuation, and the host contract of SCL, and how it was developed and frozen.

Chapter 4: Experimental Results and Performance Evaluation

This chapter describes the datasets, evaluation metrics, and implementation environment, and presents the results of both controllers: AC-MOT on VisDrone2019-MOT and UAVDT, and SCL on MOT17, KITTI, and UAVDT under detector and score changes. It compares the proposed controllers with existing approaches and discusses the results.

Chapter 5: Conclusions and Future Work

This chapter summarizes the findings of the thesis and outlines directions for future work.

Chapter 2

Background and Literature Review

2.1 Introduction

This chapter reviews the field in which the thesis is placed. Section 2.2 gives an overview of multi-object tracking: its evolution, its applications and challenges, its real-time constraints, a formal definition of tracking by detection, and its main paradigms. Section 2.3 reviews object detectors and the meaning and calibration of their confidence scores, and Section 2.4 reviews the tracking-by-detection trackers used in this work and how they use these scores. Section 2.5 introduces the datasets of the field. Section 2.6 reviews the work closest to the two proposed controllers, summarizes it in a comparison table, and states the gap that the thesis addresses. Sections 2.2–2.5 extend the survey published by the author in [18].

2.2 Multi-Object Tracking Overview

2.2.1 Evolution of Object Detection and Tracking

Object detection and tracking have evolved through three technological phases [19]. Before 2012, detectors relied on hand-engineered features such as the histogram of oriented gradients [20] and scale-invariant keypoints [21], combined with sliding windows and shallow classifiers. The second phase began when convolutional neural networks (CNNs) trained on large labelled datasets [22, 23] outperformed hand-crafted features; region-based detectors [24, 25] and single-stage detectors [26, 27] pushed the accuracy–speed frontier, and deep residual networks [28] became standard backbones. The third phase, beginning around 2020, introduced attention-based models [29, 30]. The detection transformer (DETR) [31] removed anchors and NMS, and transformer-based trackers [32–34] integrated detection and association in one network.

Throughout these phases, multi-object tracking has been dominated by tracking by detection, whose trackers consume the output of an independent detector [5, 6]. This thesis works within that paradigm, because it is the one in which detectors and trackers are combined and exchanged independently, and therefore the one in which their operating points must be reconciled.

2.2.2 Applications and Challenges

Real-time detection and tracking are required wherever a system must perceive its environment continuously. Surveillance and security systems detect abnormal behavior and track people and vehicles in live streams. Driver assistance and autonomous driving rely on real-time detection of pedestrians, vehicles, and obstacles [3]. UAVs apply detection and tracking in traffic monitoring, inspection, search, and defense [1, 35], and the aerospace literature has studied airborne target tracking [36, 37], onboard deep detection [38, 39], and vision-based guidance [40].

In crowded scenes, individual objects are hard to separate, and occlusion causes missed detections and identity switches. Fast motion, deformation, and long disappearance require motion models and appearance cues that remain reliable over many frames. Small objects, common in aerial footage, produce few pixels and low detector confidence [8]. Illumination changes, blur, and weather reduce the detector's recall and change the distribution of its confidence scores [41]. Multi-object tracking therefore faces three coupled problems: detecting the objects, associating detections across frames, and keeping identities through occlusion [19].

2.2.3 Real-Time Constraints

Real-time performance is limited by three factors [35]. The first is the accuracy–speed trade-off: more accurate models generally need more computation and longer inference. The second is the hardware, which ranges from data-center graphics processing units (GPUs) to low-power embedded processors on board a vehicle or an aircraft. The third is the deployment context: bandwidth, power supply, and the need for onboard rather than remote processing. For a tracking pipeline these constraints act mainly through the detector, whose input resolution sets most of the cost; the tracker itself is usually inexpensive. A controller that changes the detector's resolution therefore also changes the cost of the pipeline, and a controller placed between the detector and the tracker must itself be cheap.

2.2.4 Formal Definition of Tracking by Detection

Let a video be a sequence of frames $\{I_t\}_{t=1}^{T}$. A detector estimates, for each frame, a set of candidates

$$D_t = \left\{ \left(x_{t,i}, s_{t,i}, c_{t,i} \right) \right\}_{i=1}^{n_t}, \tag{2.1}$$

where $\mathbf{x}_{t,i}$ is a bounding box, $s_{t,i} \in (0, 1)$ a confidence score, and $c_{t,i}$ a class label. Multi-object tracking associates detections across time into trajectories $T=\{\tau_j\}$, each a sequence of boxes between a start frame and an end frame. An online tracker solves, frame by frame, an assignment problem

$$\pi_t^{\iota} = \arg\min_{\pi_t} \sum_{(i,j)\in\pi_t} C\left(x_{t,i}, \tau_j\right), \tag{2.2}$$

where π_t is a one-to-one matching between the detections of frame t and the existing tracks, and C is a cost based on predicted motion, appearance, or learned embeddings. The Hungarian method [42] solves this assignment exactly, and a Kalman filter [43] is the usual motion predictor.

Two families of thresholds enter this formulation, and they are the subject of this thesis. The detector keeps only candidates whose score exceeds its confidence threshold and suppresses candidates that overlap a higher-scoring candidate by more than its NMS threshold, after resizing the frame to its input resolution. The tracker then admits detections to association and to track birth only above its own score thresholds. The quality of T therefore depends not only on the detector and the tracker, but also on whether their thresholds fit the scenes and the score distribution that the detector actually produces.

2.2.5 Multi-Object Tracking Paradigms

Tracking by detection. A detector is applied independently to every frame, and an online tracker associates the detections over time using motion and appearance cues [5, 44]. The paradigm benefits directly from advances in detectors and remains modular: any detector can feed any tracker. It is the paradigm used throughout this thesis.

Joint detection and tracking. JDE learns detection and appearance embeddings in a shared model [45]; FairMOT treats detection and re-identification (ReID) as equally important tasks with an anchor-free design [46]; CenterTrack predicts object centers and their displacements between frames [47]. These methods reduce the cost of a separate appearance model, but they couple the detector and the tracker, so that neither can be exchanged independently.

Transformer-based tracking. TransTrack propagates queries between frames [32]; TrackFormer separates object queries for new objects from track queries for existing ones [33]; MOTR carries track queries across frames and updates them by attention [34]; MOTIP predicts identities directly [48]. These

methods are fully differentiable, but they remain computationally expensive, and their thresholds are internal to the network.

2.3 Object Detectors

Object detectors are broadly divided into two-stage and single-stage architectures, with transformer-based detectors forming a third group. Table (2.1) summarizes the detectors discussed in this section.

2.3.1 Two-Stage Detectors

The region-based CNN family first proposes candidate regions and then classifies and refines them [24]. Faster R-CNN introduced a region proposal network that shares convolutional features with the detection head, which made the proposals nearly free and improved both speed and accuracy [25]. Mask R-CNN added a parallel branch that predicts a segmentation mask for each detection [49]. Two-stage detectors are accurate, particularly for small objects and cluttered scenes, but they are generally slower than single-stage detectors.

2.3.2 Single-Stage Detectors

Single-stage detectors predict boxes and class probabilities in one pass over the image. The YOLO (You Only Look Once) family treats detection as a regression problem over a grid [26]; successive versions improved the backbone, the neck, and the training recipe [50–53], and recent generations distributed as software libraries, such as YOLOv8 [54] and YOLO11 [55], offer a range of model sizes. The Single Shot MultiBox Detector (SSD) predicts boxes from feature maps at several scales [27]. RetinaNet introduced the focal loss to address the imbalance between foreground and background during the training of dense detectors [56]. EfficientDet combined a bidirectional feature pyramid with compound scaling of resolution, depth, and width [57]. Feature pyramid networks [58] are the standard architectural response to scale variation in all of these detectors.

2.3.3 Transformer-Based Detectors

DETR formulates detection as set prediction with a transformer encoder–decoder, which removes anchors and NMS [31]. The original DETR converges slowly and is expensive. Real-time transformer detectors such as RT-DETR [59] and RF-DETR [60] keep the set-prediction formulation while reducing latency to the range of single-stage detectors.

Table (2.1) Detector families discussed in this thesis.

The last column indicates the thresholds that the user chooses when the detector is run.

Detector	Year	Architecture	Key idea	User thresholds
R-CNN [24]	2014	two-stage	CNN features on region proposals	confidence, NMS
Faster R-CNN [25]	2017	two-stage	shared region proposal network	confidence, NMS
Mask R-CNN [49]	2017	two-stage	parallel mask branch	confidence, NMS
YOLO [26]	2016	single-stage	grid regression of boxes and classes	confidence, NMS
SSD [27]	2016	single-stage	multi-scale default boxes	confidence, NMS
RetinaNet [56]	2017	single-stage	focal loss for class imbalance	confidence, NMS
EfficientDet [57]	2020	single-stage	bidirectional pyramid, compound scaling	confidence, NMS
YOLOX [52]	2021	single-stage	anchor-free, decoupled head	confidence, NMS
DETR [31]	2020	transformer	set prediction, no NMS	confidence
RT-DETR [59]	2024	transformer	real-time hybrid encoder	confidence

2.3.4 Detector Confidence and Calibration

A classifier is calibrated when its confidence equals its probability of being correct. Modern neural networks are often poorly calibrated [61], and object detectors are no exception: their confidence depends on the position and size of the box as well as on the class, and it shifts with the data [9]. Detectors of different families [25, 26, 31, 53, 59] produce different score distributions, and soft suppression changes the score of overlapping boxes [62]. The same object may therefore receive 0.9 from one detector and a much lower score from another. Calibration methods fit a mapping from scores to probabilities on labelled data [9]. That is not possible when a detector is exchanged in a deployed pipeline without labels. Test-time adaptation methods adapt a model to unlabelled data [63], but they change the model's weights, whereas this thesis keeps both the detector and the tracker frozen.

Classical image thresholding offers a different tool. Otsu's method [17] chooses, without labels, the threshold that best splits a histogram into two groups. For a candidate threshold k, let $\omega_0(k)$ and $\omega_1(k)$ be the fractions of values below and above k, and $\mu_0(k)$ and $\mu_1(k)$ their means. The method selects

$$k^{i} = \arg\max_{k} \omega_0(k)\,\omega_1(k)\,(\mu_0(k) - \mu_1(k))^2, \tag{2.3}$$

the threshold that maximizes the between-class variance. It needs no labels and adapts to the shape of the distribution. The self-calibrating layer of Chapter 3 applies it to the logits of recent detector scores.

2.4 Tracking-by-Detection Trackers

This section reviews the trackers that are used or discussed in the thesis, with attention to how each one uses detector scores. Table (2.2) summarizes them.

2.4.1 Motion-Based Trackers

SORT combines a constant-velocity Kalman filter with Hungarian assignment on box overlap [5]. It is simple and fast, but it relies on motion alone, so occlusion and irregular motion cause many identity switches. The IOU tracker shows how far pure overlap association can go at high frame rates [64]. ByteTrack introduced a second association step: detections above a high threshold are associated first, and the tracks that remain unmatched are then associated with low-confidence detections, which recovers partially occluded objects that other trackers discard [6]. OC-SORT refines the Kalman filter with an observation-centric re-update and a velocity-direction consistency term, which improves robustness to occlusion and non-linear motion [65]; it uses a single confidence threshold. Buffered IoU matching enlarges the boxes during association to handle irregular motion [66].

2.4.2 Motion and Appearance Trackers

DeepSORT adds deep appearance embeddings to SORT and combines the Kalman prediction with the cosine distance between embeddings, which reduces identity switches [44]. BoT-SORT adds camera-motion compensation, an improved Kalman state, and ReID features to the ByteTrack association [67]. StrongSORT revisits DeepSORT with an exponential moving average of appearance features, camera-motion compensation by the enhanced correlation coefficient [68], and other refinements [69]. Deep OC-SORT adds adaptive appearance re-identification to OC-SORT [70], and SMILEtrack learns similarity for occlusion-aware association [71]. Many recent trackers improve association further with trained or domain-specific parts [72–77]; these change or retrain the tracker.

2.4.3 Recent Trackers Used in This Thesis

Hybrid-SORT uses weak cues, such as the confidence state and the height of a box, alongside strong cues [78]. SparseTrack decomposes crowded scenes by pseudo-depth before association [79]. BoostTrack boosts the confidence of likely true detections and augments the similarity measure [80]. PD-SORT adds pseudo-depth cues to the motion model to handle occlusion [81]. U2MOT is an uncertainty-aware tracker trained without identity labels [82]. OATrack [83] is a recent tracker for small objects in UAV video; it weights association by detection quality, uses a YOLO detection cache, and has a fixed tracker-side confidence gate.

2.4.4 How Trackers Use Detector Scores

All of these trackers read detector scores through fixed thresholds that their authors tuned for one detector, usually YOLOX [52] on MOT17. Two kinds of trackers can be distinguished (Fig. (2.1)). A *single-stage* tracker, such as OC-SORT, ignores every box below one threshold. A *two-stage* tracker, such as ByteTrack, uses boxes above a high threshold to match tracks and boxes above a higher threshold to start tracks, and it uses lower-score boxes, down to a low threshold, in a second step that can only continue tracks left unmatched by the first step. If a detector gives scores on a different scale from the one the thresholds were tuned for, a single-stage tracker can lose most of its boxes, and a two-stage tracker can lose its ability to start tracks. The published comparison in [18] shows the steady progress of tracking accuracy from motion-only methods to trackers that combine motion, appearance, and low-score association; it does not, and no leaderboard does, show how a tracker behaves when its detector changes.

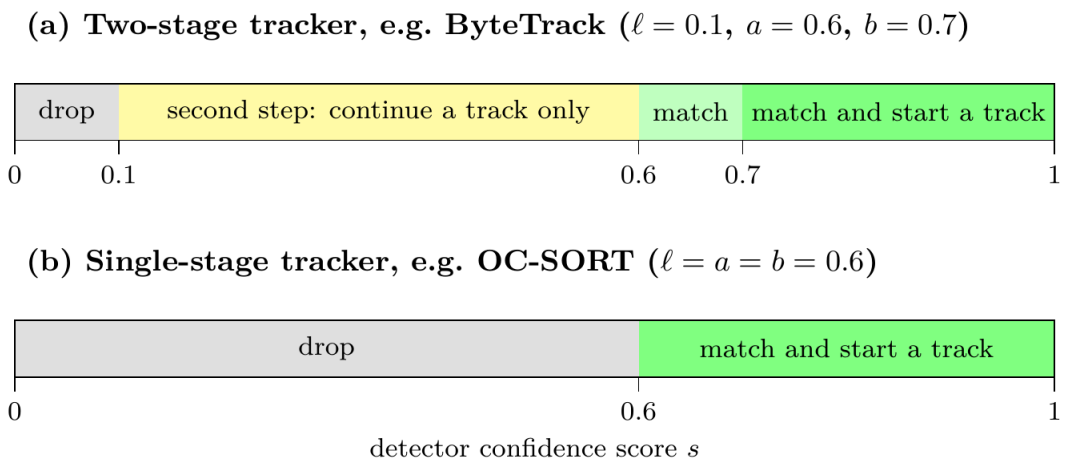


Fig. (2.1) How a two-stage and a single-stage tracker use the detector score, with the published thresholds of ByteTrack (official setting) and OC-SORT.

ℓ: lowest score used; a: score needed to match a track; b: score needed to start a track.

Table (2.2) Tracking-by-detection trackers discussed in this thesis and how they use detector scores.

AC-MOT and SCL are the two controllers proposed in Chapter 3.

Tracker	Year	Main cues	Score use	Used with
SORT [5]	2016	Kalman motion, IoU	one threshold	background
DeepSORT [44]	2017	motion, appearance	one threshold	background
ByteTrack [6]	2022	motion, IoU, low-score recovery	two stages	AC-MOT, SCL
BoT-SORT [67]	2022	motion, camera motion, ReID	two stages	SCL
OC-SORT [65]	2023	observation-centric motion	one threshold	SCL
StrongSORT [69]	2023	motion, appearance, camera motion	one threshold	background
U2MOT [82]	2023	uncertainty-aware association	published static setting	AC-MOT development
Hybrid-SORT [78]	2024	strong and weak cues	two stages	SCL

Tracker	Year	Main cues	Score use	Used with
BoostTrack [80]	2024	confidence boost, similarity	two stages	SCL
SparseTrack [79]	2025	pseudo-depth decomposition	two stages	AC-MOT development, SCL
PD-SORT [81]	2025	pseudo-depth motion	one threshold	SCL
OATrack [83]	2026	quality-weighted association	confidence gate	AC-MOT

2.5 Datasets for Detection and Tracking

Detection models are commonly trained on COCO, which contains more than 330,000 images with annotations for 80 object categories [84], and ImageNet provides the large-scale classification data on which most backbones are pretrained [23]. For tracking, the MOTChallenge benchmarks MOT16 and MOT17 [85, 86] contain pedestrian sequences from static and moving cameras, and MOT20 contains very crowded scenes [87]. KITTI provides driving sequences with car and pedestrian tracks [3]. DanceTrack contains dancers with uniform appearance and diverse motion, which makes association by appearance difficult [88]. For aerial tracking, VisDrone2019 provides drone video captured at different altitudes, viewpoints, and illumination [1], and UAVDT provides vehicle tracking from UAVs under varied weather, altitude, and camera views [7]. The datasets and splits used in this thesis are described in Chapter 4.

2.6 Literature Review

Four lines of work relate to the proposed controllers: adaptive inference methods, which change how a detector is running; adaptive threshold methods, which change settings inside a tracker; aerial tracking methods, which address the difficulties of UAV video inside the tracker or the detector; and the optimization and evaluation tools used to develop and test the controllers.

2.6.1 Adaptive Inference for Object Detection

Dynamic neural networks change their structure or parameters for each input [89], and resolution-adaptive networks spend less computation on easy inputs [90]. AdaScale selects the input scale of each video frame with a scale regressor trained on detector features [11]. ApproxDet switches detection configurations at run time with accuracy and latency models trained offline [12]. Chameleon re-profiles the configuration of video analytics pipelines over time [13]. These methods trade detection accuracy against cost, and they select a configuration with additional trained models or periodic profiling. They are designed for detection or video analytics, not for MOT, where a detection error can also break an association or switch an identity.

2.6.2 Adaptive Thresholds in Multi-Object Tracking

Fixed tracker thresholds decide which detections are accepted and how they are associated, but their reliability changes with object density, occlusion, and detection confidence, so several methods adapt them. Adaptive ByteTrack sets ByteTrack's confidence threshold from the detections of each frame [14], and AdapTrack adapts the matching thresholds of its data association [15]. ACT sets the high and low thresholds from the first quartile and the number of the current scores [16]. ConfTrack weights the Kalman update and the matching cost by the score [91], UTrack uses the spread of the boxes merged by NMS [92], and PlugTrack blends motion predictors with a trained network [93]. These methods are built into one tracker, use only the current frame, or need trained parts.

2.6.3 Aerial Multi-Object Tracking

The VisDrone [1] and UAVDT [7] benchmarks established large annotated drone video collections and documented the characteristic difficulties of the domain: small objects, high density, changing altitude and viewpoint, and illumination changes. Surveys of deep learning for drone-based detection and tracking [2] and of computer vision for UAVs [94] summarize the resulting methods. UAVMOT addresses the motion of the UAV camera inside the tracker [95], and appearance-aided joint probabilistic data association has been used for multiclass MOT in cluttered aerial images, mainly to reduce identity switches and track fragmentation [96]. Small objects are a central difficulty: detectors that keep features at high resolution have been designed for small objects on the airport surface [97] and for tiny objects in remote sensing imagery [98], and LSO-YOLO designs a lightweight real-time detector for small UAV objects [99]. OATrack weights association by detection quality for small UAV objects [83]. These works improve the tracker or the detector itself; the input resolution, the confidence threshold, and the NMS threshold of the detector remain fixed settings.

2.6.4 Optimization and Evaluation Tools

Detector and tracker parameters affect accuracy, identity consistency, and speed in conflicting ways, so they need systematic search and an evaluation that covers both detection and identity. Optuna provides define-by-run search with the tree-structured Parzen estimator (TPE) sampler [100], which was used during the development of AC-MOT on calibration data only. Tracking is evaluated with the CLEAR MOT metrics [101], the identity measures [102], and HOTA [10], computed with the official MOTChallenge tools [86], and the reliability of a difference is judged with bootstrap resampling [103].

2.6.5 Summary of the Literature and Research Gap

Table (2.3) summarizes the methods closest to this thesis. The review points to a gap between how tracking methods are developed and how they are deployed. Detectors and trackers are developed and benchmarked with fixed operating points tuned for one detector and one kind of scene. In deployment, scenes change within a video, and detectors are exchanged more often than trackers are retuned. Adaptive inference systems learn or profile a policy for each model and target detection, adaptive-threshold trackers adapt one tracker to the current frame, and calibration needs labels. What is missing is (i) a training-free way to control the detector operating point online in changing aerial scenes, judged with tracking metrics, and (ii) a training-free and tracker-agnostic way to keep published trackers working when the detector's score scale changes, which leaves a tracker unchanged when nothing needs to be changed. The two controllers of Chapter 3 address these two points.

Table (2.3) Comparison table of the literature review: methods closest to the proposed controllers.

Year	Method	Contribution	Adapted part	Limitation
2018	Chameleon [13]	re-profiles the configuration of video analytics pipelines over time	pipeline configuration	periodic profiling; video analytics, not MOT
2019	AdaScale [11]	selects the input scale of each frame with a scale regressor trained on detector features	detector input scale	trained regressor; detection, not tracking
2020	ApproxDet [12]	switches detection configurations with accuracy and latency models trained offline	detector configuration	models trained offline; detection, not tracking
2020	RANet [90]	resolution-adaptive network that spends less computation on easy inputs	network resolution	changes and trains the network
2020	Multivariate calibration [9]	calibrates detector confidence with box position and size	detector scores	needs labelled data
2022	UAVMOT [95]	models the motion of the UAV camera inside the tracker	tracker motion model	changes the tracker; detector settings fixed
2023	Adaptive ByteTrack [14]	sets ByteTrack's confidence threshold from the detections of each frame	tracker confidence threshold	built into one tracker; current frame only
2024	AdapTrack [15]	adaptive thresholding in the matching step	matching thresholds	built into one tracker
2024	ConfTrack [91]	weights the Kalman update and the matching cost by the score	Kalman update, matching cost	built into one tracker
2024	AF-JPDA [96]	appearance-aided probabilistic association for multiclass aerial MOT	data association	changes the tracker; detector settings fixed
2025	UTrack [92]	uses the spread of the boxes merged by NMS as detection uncertainty	association	built into one tracker
2026	ACT [16]	sets high and low thresholds from the first quartile and the number of the current scores	tracker high and low thresholds	current frame only; fixed score levels
2026	PlugTrack [93]	blends motion predictors with a trained network	motion prediction	needs trained parts
2026	OATrack [83]	quality-weighted association for small UAV objects	association weights	fixed confidence gate; detector settings fixed

Chapter 3

Proposed Training-Free Adaptive Control Approach

3.1 Introduction

This chapter presents the two controllers proposed in this thesis. Both are small, rule-based programs that run beside a frozen detector and a frozen tracker, use only information that is available at run time, and need no training and no labels. Section 3.2 presents the scene-adaptive controller of the detector operating point, AC-MOT, and Section 3.3 explains how optimization experiments on calibration data led to its frozen values. Section 3.4 presents the self-calibrating control layer, SCL, which adapts the input and thresholds of the tracker to the detector's score distribution, and Section 3.5 describes its development and freeze. Section 3.6 summarizes the chapter. The evaluation of both controllers is given in Chapter 4.

3.2 Scene-Adaptive Detector Operating-Point Controller (AC-MOT)

AC-MOT is a small, rule-based controller that runs next to the detector. On frame 1 and then on every tenth frame, it measures cheap cues of the current image and of the boxes tracked in the previous frame (Section 3.2.2), averages the recent crowd level into a scene complexity index (SCI) (Section 3.2.3), chooses a target state (LOW, MEDIUM, or HIGH), adjusts it with a recovery probe and hysteresis and dwell rules (Section 3.2.4), and maps the state to one of three fixed detector operating points (Section 3.2.5). Section 3.2.6 explains how each value is chosen.

3.2.1 Overview and Operating Point

Figure (3.1) shows where AC-MOT sits in the pipeline: the controller chooses the detector settings before the detector runs, and the detector and the tracker then run as usual. The detector settings at frame are called the operating point

$$\theta_t = (c_t, \eta_t, r_t), \tag{3.1}$$

which has three parts:

- r_t is the input resolution, the long side of the resized image in pixels. A larger input shows small objects better but takes more time.

- η_t is the NMS IoU threshold. When two boxes overlap with an intersection over union (IoU) above η_t, the box with the lower score is removed. A lower η_t removes more overlapping boxes.
- c_t is the score threshold. Boxes with a detector score below c_t are removed before they reach the tracker. A higher c_t passes fewer but more reliable boxes.

The detector weights and the tracker are frozen; AC-MOT changes only θ_t.

The controller uses only information that is available at run time:

$$\theta_t = \pi\left(I_t, B_{1:t-1}\right), \tag{3.2}$$

where π is the control rule described in this section, I_t is the image of the current frame, and $B_{1:t-1}$ are the boxes that the tracker reported for the earlier frames 1 to $t-1$. The tracker output of frame t is used only from frame $t+1$ on. No ground truth, no future frame, and no detector score enters the decision.

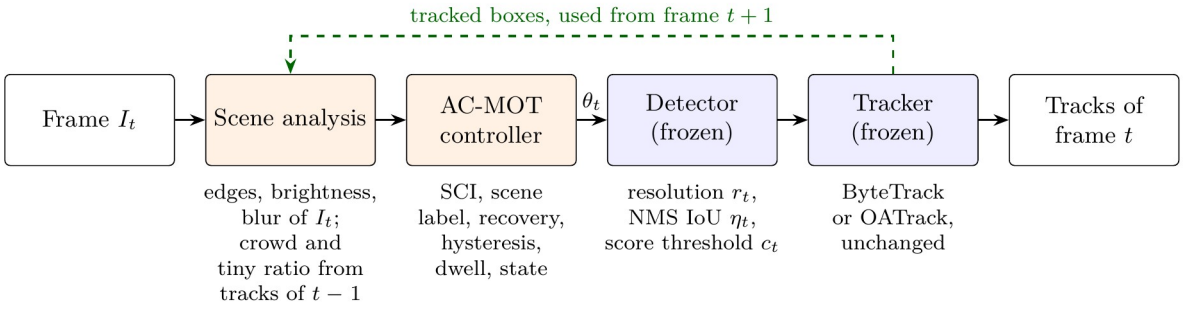


Fig. (3.1) AC-MOT in a tracking-by-detection pipeline.

Scene analysis and the controller choose the operating point $\theta_t=(c_t, \eta_t, r_t)$ before the detector runs. The frozen detector and tracker are not modified. Tracked boxes of frame t feed the scene analysis of later frames (dashed arrow).

3.2.2 Scene Cues

The controller needs a cheap description of the scene. Scene analysis therefore does not run on every frame. It runs on frame 1 and then once every 10 frames (t=1 or t mod 10=1, that is, frames 1, 11, 21, …). Each of these frames is an analysis step j. Between two analysis steps the controller keeps its last decision. Two kinds of cues are computed at each step: image cues, which describe how the current frame looks, and box cues, which describe the objects that the tracker found in the previous frame.

Image cues of the current frame. The frame is reduced to a quarter of its width and height and converted to gray levels, so that the cues are cheap to compute. Three cues are then measured:

- the edge density e_t, the fraction of pixels marked as edges by the Canny detector (lower and upper thresholds 50 and 120); many edges indicate a cluttered scene;
- the brightness β_t, the mean gray level from 0 (black) to 255 (white); a low value indicates a dark or night scene;
- the sharpness v_t, the variance of the Laplacian of the gray image [104]; a low value indicates a blurred frame.

Box cues of the previous frame. Let N_j be the number of boxes that the tracker reported for frame $t-1$, and let a_i be the area of box i in pixels of the original image. The crowd cue and the tiny-object ratio are

$$C_j = min\left(\frac{N_j}{30}, 1\right), \tag{3.3}$$

$$T_j = \frac{1}{N_j}\sum_{i=1}^{N_j} 1\left[a_i < 32^2\right]. \tag{3.4}$$

C_j is the number of tracked objects divided by 30 and capped at 1, so every frame with 30 or more objects has $C_j=1$. T_j is the fraction of tracked boxes smaller than 32×32 pixels, which is the size limit of small objects in the COCO benchmark [84]. The indicator $\mathbf{1}[\cdot]$ is 1 when its condition holds and 0 otherwise, and $T_j=0$ when $N_j=0$.

3.2.3 Scene Complexity Index and Scene Label

The SCI measures how crowded the recent frames are. A single frame is a noisy measurement, because the tracker may miss or add a few objects in any frame, so the SCI averages over time. It is the mean crowd cue of the last seven analysis steps:

$$S_j = \frac{1}{\left|H_j\right|}\sum_{k \in H_j} C_k, \tag{3.5}$$

where H_j holds the steps $j-6, \ldots, j$ (fewer at the start of a video) and $|H_j|$ is their number. With a stride of 10 frames, the seven steps cover about the last 60 frames, so a single unusual frame changes S_j only a little. Because $C_j \le 1$, S_j lies between 0 and 1.

The SCI uses the crowd cue alone, selected by the cue audit of Section 3.3.4. The other four cues are still used as checks on the state, mainly through the scene label

$$\ell_j = \begin{cases} \text{night} & \text{if } \beta_t < 80, \\ \text{blur} & \text{else if } v_t < 180, \\ \text{tiny} & \text{else if } T_j > 0.50, \\ \text{crowded} & \text{else if } C_j > 0.65 \text{ or } e_t > 0.13, \\ \text{clear} & \text{otherwise.} \end{cases} \tag{3.6}$$

The conditions are checked from top to bottom, and the first one that holds gives the label. For example, T_j>0.50 means that more than half of the tracked boxes are tiny, and C_j>0.65 means that at least 20 objects are tracked.

Only the crowd cue enters the SCI, and it also enters the crowded label. The tiny-object ratio T_j is used by the state rules of Section 3.2.4 and gives the tiny label, and the edge density e_t is used only through the crowded label. The brightness β_t and the sharpness v_t act only through the night and blur labels, which do not lead to MEDIUM or HIGH by themselves: because they are checked first, a dark or blurred frame cannot receive the crowded or tiny label, while the SCI conditions and the tiny-object guard of Eq. (3.7) still apply.

3.2.4 Three-State Controller

The controller is always in one of three states: LOW, MEDIUM, or HIGH. The names describe the scene: LOW for a sparse scene, HIGH for a crowded scene or a scene of mostly tiny objects. They do not rank the input size: in the final action map the HIGH state uses the smallest input (Table (3.1)). The state is updated only at analysis steps, in four steps (Algorithm 3.1). Step 1 proposes a target state from the cues. Steps 2–4 adjust this target so that the controller reacts to a sudden drop in the object count but does not switch back and forth.

Step 1: target state. The target is the state that the current cues ask for. It is given by the SCI, the tiny-object ratio, and the scene label:

$$g_j = \begin{cases} \text{HIGH} & \text{if } S_j > \tau_h \text{ or } T_j > 0.50, \\ \text{MEDIUM} & \text{else if } S_j > \tau_m \text{ or } \ell_j \in \{\text{crowded}, \text{tiny}\}, \\ \text{LOW} & \text{otherwise,} \end{cases} \tag{3.7}$$

where τ_m=0.297 and τ_h=0.511 are the two SCI thresholds (Table (3.2)). Since S_j is the recent average number of tracked objects divided by 30, the SCI conditions give MEDIUM above about 9 objects ($30\tau_m$=8.9) and HIGH above about 15 objects ($30\tau_h$=15.3).

Step 2: recovery probe. The SCI averages seven steps, so it reacts slowly when the tracker suddenly reports far fewer objects than a few steps earlier. The recovery probe responds to such a collapse of the object count. Instead of letting the falling count move the controller toward a lower state, it sets the target to HIGH for a short time. To detect a collapse, it keeps a slowly decaying peak P_j of the object count and a drop counter d_j:

$$d_j = \begin{cases} d_{j-1} + 1 & \text{if } P_{j-1} \geq 5 \text{ and } N_j < 0.40\, P_{j-1}, \\ 0 & \text{otherwise,} \end{cases} \tag{3.8}$$

$$P_j = max\left(N_j, \lfloor 0.8\, P_{j-1} \rfloor\right), \tag{3.9}$$

with $P_0 = d_0 = 0$. A collapse is detected when at least 5 objects have been seen before and fewer than 40% of them remain. The peak loses 20% at each step, so an old peak is forgotten after a few steps. While $1 \leq d_j \leq 3$, the target g_j is set to HIGH, so the probe lasts at most three analysis steps (30 frames); the dwell rule of Step 4 still applies.

Step 3: hysteresis. Without a margin, a scene whose SCI stays close to a threshold would make the controller switch at almost every step. Hysteresis prevents this: a move to a lower state is held while the scene is still close to difficult:

$$g_j \leftarrow \begin{cases} \text{HIGH} & \text{if } q_{j-1} = \text{HIGH and } \left(S_j > 0.50 \text{ or } T_j > 0.40\right), \\ \text{MEDIUM} & \text{if } q_{j-1} = \text{MEDIUM}, g_j = \text{LOW and } S_j > 0.25, \\ g_j & \text{otherwise.} \end{cases} \tag{3.10}$$

The hold values lie just below the values that lead into each state ($0.50 < \tau_h$, 0.40 below the tiny guard of 0.50, and $0.25 < \tau_m$). Upward moves are not held.

Step 4: dwell. The dwell rule sets a minimum time between two changes. After a change, the state is kept for at least 30 frames, that is, three analysis steps:

$$q_j = \begin{cases} g_j & \text{if } t - t_c \geq 30, \\ q_{j-1} & \text{otherwise,} \end{cases} \tag{3.11}$$

where t_c is the frame of the last state change. The controller starts in LOW with $t_c = 1$, so the first change is possible at frame 31. Whenever $q_j \neq q_{j-1}$, t_c is set to t.

Why changes are kept rare. Each change of state changes the boxes that reach the tracker, so the temporal rules let the state follow lasting changes of the scene, not single frames: the seven-step window smooths the SCI over about 60 frames, the holds add a margin below each threshold, and the dwell rule allows at most one change every 30 frames.

Example. Consider a clear, sharp video in which the tracker reports 20 objects of normal size in every frame (C_j=0.67), and only 10 objects from frame 71 on. The controller starts in LOW. The first step sees no earlier boxes (C_j=0), so S_j grows as the window fills: 0.33 at frame 11, 0.44 at frame 21, and 0.50 at frame 31. The target is MEDIUM from frame 11, but the dwell rule keeps LOW until frame 31, where the state becomes MEDIUM. From frame 41, S_j exceeds τ_h and the target is HIGH; the dwell rule delays this change to frame 61. When the scene thins, S_j falls slowly because the window still holds the earlier steps, so the state stays HIGH until frame 101. At frame 111, S_j=0.48 is below both τ_h and the hold value of 0.50, and the state returns to MEDIUM. It then stays in MEDIUM, because 10 tracked objects keep S_j at about 0.33, above τ_m.

The state q_j then selects the operating point from the action map, and the detector uses it until the next analysis step.

Input: frames $I_1, I_2, \ldots$; frozen detector and tracker; constants of Table (3.2)
Output: tracks of every frame

$j \leftarrow 0$; $q_0 \leftarrow$ LOW; $P_0 \leftarrow 0$; $d_0 \leftarrow 0$; $t_c \leftarrow 1$
for each frame $t = 1, 2, \ldots$ **do**
  if $t = 1$ **or** $t \bmod 10 = 1$ **then** ▷ analysis step, every 10 frames
    $j \leftarrow j + 1$
    $e_t, \beta_t, \nu_t \leftarrow$ image cues of I_t ▷ Section 3.2.2
    $N_j, C_j, T_j \leftarrow$ cues of the tracked boxes of frame $t-1$ ▷ no boxes at $t = 1$
    $S_j \leftarrow$ mean of C_k over the last seven steps $k \in H_j$ ▷ SCI, Eq. (3.5)
    $\ell_j \leftarrow$ label, Eq. (3.6); $g_j \leftarrow$ target, Eq. (3.7) ▷ Step 1: target
    update d_j and P_j, Eqs. (3.8)–(3.9)
    if $1 \le d_j \le 3$ **then** $g_j \leftarrow$ HIGH ▷ Step 2: recovery probe
    end if
    apply hysteresis to g_j, Eq. (3.10) ▷ Step 3: hysteresis
    if $t - t_c \ge 30$ **and** $g_j \ne q_{j-1}$ **then** ▷ Step 4: dwell
      $q_j \leftarrow g_j$; $t_c \leftarrow t$
    else
      $q_j \leftarrow q_{j-1}$
    end if
  end if
  $(r_t, \eta_t, c_t) \leftarrow$ action of state q_j, Table (3.1) ▷ kept until the next analysis step
  detect with r_t and η_t; drop boxes with score $< c_t$
  update the tracker; store its boxes of frame t ▷ used from frame $t+1$
end for

Algorithm 3.1. AC-MOT controller for one video.

3.2.5 Action Map

Table (3.1) gives the operating point of each state. The actions are not set by hand: they are selected together with τ_m and τ_h by the joint search on the calibration sequences, whose objective is the change in HOTA, a metric that balances detection and association (Section 3.2.6).

In LOW, the sparse state, the detector uses the largest input, loose NMS, and a low score threshold, so it passes more weak boxes. In MEDIUM and HIGH it passes only boxes with a score of at least 0.40. The HIGH state, used for crowded scenes and for scenes of mostly tiny objects, thus has the smallest input. This may look surprising, since a larger input shows small objects better, but the search optimizes tracking quality, not the detection of every small object: a score threshold of 0.40 gives the tracker fewer but more reliable boxes (Section 3.3.4), and the smaller input costs less detector time. Chapter 4 shows that the main effect of AC-MOT is a more selective detection stream: fewer false positives and identity switches, with similar or slightly lower recall. This action is the result of the search, not a general rule that crowded scenes need smaller inputs.

The score threshold c_t is applied on the detector side before the tracker. It is separate from any confidence gate inside the tracker, such as the fixed tracker-side gate of OATrack.

Table (3.1) Action map of AC-MOT.

State names describe scene difficulty, not a resolution rank. Selected by the joint search on the calibration sequences.

State	Resolution r (pixels)	NMS IoU η	Score threshold c
LOW	1536	0.70	0.10
MEDIUM	1280	0.45	0.40
HIGH	1088	0.60	0.40

3.2.6 Selection of Parameter Values

All values of Tables (3.1) and (3.2) are fixed before the final evaluation and are not changed after it. The last column of Table (3.2) gives the source of each value. There are four sources.

Joint search on the calibration sequences. The SCI thresholds τ_m and τ_h and the action map are selected together by an Optuna TPE search with 60 trials [100]; the search ranges are given in Section 3.3.4. The calibration sequences are eight sequences of the VisDrone2019-MOT training split. They serve as the validation (tuning) data of the final stage and are disjoint from the evaluation sequences. Each trial is scored by four-fold cross-validation (CV), with two sequences held out in each fold, as the mean change in HOTA against the baseline. The candidate values come from existing settings. The input sizes (1088, 1280, and 1536 pixels) and NMS values (0.45, 0.60, and 0.70) are the operating points for which detector outputs are cached; 1536 pixels with NMS 0.70 is the baseline point, and 0.70 is the default NMS value of the Ultralytics detector. The three score thresholds are the low and high thresholds of ByteTrack (0.10 and 0.25) and the confidence gate of OATrack (0.40).

Cue audit on the calibration sequences. The choice of the crowd cue as the only SCI input comes from testing all 31 non-empty subsets of the five cues with the same cross-validation (Section 3.3.4).

Temporal sweep on the validation split. The analysis stride and the SCI window are selected in the first development stage, with the YOLOv8n pipeline, on the VisDrone2019-MOT validation sequences. The sweep tests 25 combinations: windows of 1, 3, 5, 7, and 9 analysis steps and strides of 1, 5, 10, 15, and 20 frames. Five of the 25 settings meet both constraints of that stage, at least 25 frames per second (FPS) and at most 271 identity switches (the number of the original controller on the same data). Among them, a window of 7 and a stride of 10 give the highest MOTA, 18.165 against 18.044 for the next setting.

Operating-point sweeps on the same data cover input resolutions from 512 to 960 pixels, confidence thresholds from 0.05 to 0.50, and NMS thresholds from 0.30 to 0.80, and define the supported values of the first optimized profiles (Section 3.3).

Fixed design values. The tiny-object area follows the COCO definition of small objects [84]. The remaining values (the crowd normalizer, the Canny thresholds, the label thresholds, the recovery, hold, and dwell values, and the initial state) come from the original AC-MOT controller and are kept unchanged in every search of the final stage, so the searched values are matched to them. For example, the crowd normalizer of 30 only sets the scale of S_j: τ_m and τ_h are searched on this scale and correspond to about 9 and 15 tracked objects.

Table (3.2) Values of the AC-MOT controller and how each value is chosen.

Calibration sequences: eight VisDrone2019-MOT training sequences used as tuning data, disjoint from the evaluation sequences. CV: four-fold cross-validation.

Symbol	Meaning	Value	How the value is chosen
τ_m	SCI threshold for MEDIUM, Eq. (3.7)	0.297	joint TPE search on the calibration sequences, 60 trials, CV
τ_h	SCI threshold for HIGH, Eq. (3.7)	0.511	same joint search
(r, η, c)	operating point of each state	Table (3.1)	same joint search over {1088, 1280, 1536}×{0.45, 0.60, 0.70}×{0.10, 0.25, 0.40}
–	SCI input cue	crowd only	cue audit of all 31 cue subsets on the calibration sequences, CV
–	analysis stride	10 frames	temporal sweep of 25 settings on the validation split; highest MOTA with ≥25 FPS and ≤271 IDS
$\|H_j\|$	SCI window	7 steps	same temporal sweep
–	tiny-object area	32×32 pixels	COCO small-object limit [84]
–	crowd normalizer, Eq. (3.3)	30 boxes	original controller; sets the scale on which τ_m, τ_h are searched
–	Canny thresholds (lower, upper)	50, 120	original controller; fixed in all final-stage searches
–	night and blur labels, Eq. (3.6)	β_t<80, v_t<180	original controller; fixed in all final-stage searches
–	crowded label, Eq. (3.6)	C_j>0.65 or e_t>0.13	original controller; fixed in all final-stage searches
–	tiny label and HIGH guard, Eqs. (3.6), (3.7)	T_j>0.50	original controller; fixed in all final-stage searches
–	recovery: minimum peak, drop ratio, peak decay, steps	5, 0.40, 0.80, 3	original controller; fixed in all final-stage searches
–	holds in HIGH (S_j, T_j) and MEDIUM (S_j), Eq. (3.10)	(0.50, 0.40), 0.25	original controller; fixed in all final-stage searches
–	dwell, Eq. (3.11)	30 frames	original controller; fixed in all final-stage searches
–	initial state	LOW	original controller; fixed in all final-stage searches

3.3 Development of the AC-MOT Controller

The controller of Section 3.2 is the result of four development stages. The quality-oriented profile (Section 3.3.1) asked whether a scene index that steers the detector settings can beat a fixed default; the identity-aware profile (Section 3.3.2) made identity stability and speed explicit objectives; the transfer

studies (Section 3.3.3) tested other data and pipelines; and the modern stage (Section 3.3.4) built the frozen controller for a modern detector with separate calibration data. The first three stages are historical: they used a weighted five-cue index with interpolated confidence and NMS values and three resolution levels (the SparseTrack study used a single edge-driven NMS rule), other detectors, and other evaluation protocols. Their numbers are reported in their own tables, each against its own baseline, and are not pooled with the modern results.

3.3.1 Quality-Oriented Profile

Baseline. The reference pipeline is the pretrained YOLOv8n detector [54] followed by ByteTrack [6], both at their default settings: confidence threshold 0.25, NMS IoU threshold 0.45, input size 640 pixels, and the default ByteTrack parameters listed in Section 4.2.4. It runs with one fixed operating point for the whole video.

Method. The first optimization-derived profile kept the same detector, used a tuned ByteTrack profile, and searched only on the VisDrone2019-MOT validation split. Each of five cues was normalized with validation statistics, and the index was their weighted sum

$$S_j = \sum_{k \in \{c,t,e,n,b\}} w_k x_{k,j}, \tag{3.12}$$

where k runs over the cues (c: crowd count, t: fraction of earlier boxes below 32×32 pixels, e: edge density, n: low brightness, b: low Laplacian variance), $x_{k,j}$ is the normalized value of cue k at analysis step j, and w_k is its weight, with $w_k \geq 0$ and $\sum_k w_k = 1$. The index was averaged over seven analyses taken every ten frames. Confidence and NMS were interpolated linearly between the values at $S=0$ and $S=1$, and two sorted thresholds selected one of three resolutions (512, 928, 960 pixels).

An Optuna TPE study [100] jointly searched the five weights, the confidence and NMS end points, and the two thresholds. The resolution levels, the supported confidence values {0.25, …, 0.45}, the supported NMS values from 0.30 to 0.70, and the temporal settings were fixed beforehand by validation sweeps (Section 3.2.6). The selection rule kept trials with at least 25 FPS and at most 271 IDS (the IDS of the original controller on the same validation data) and then took the highest MOTA, with lower IDS, HOTA, IDF1, and FPS as tie-breaks. The frozen profile (archived as Trial 24) has w_c=0.1294928, w_t=0.2217477, w_e=0.4337134, w_n=0.0535577, w_b=0.1614885, confidence end points (0.30, 0.40), NMS

end points (0.35, 0.35), and thresholds (0.1353494, 0.2872868). On validation it reached MOTA 23.038, HOTA 36.110, IDF1 40.758, and IDS 270 at 37.169 FPS.

Comparison. Table (3.3) compares the profile with the baseline on the historical VisDrone test-dev split, with paired bootstrap intervals (5000 resamples, seed 42). A scene index that steers the detector settings improved tracking quality over the fixed default point, with HOTA, MOTA, and IDF1 intervals above zero. The IDS interval, [−114, 219], includes zero; the next stage made identity stability an explicit objective. The final controller keeps two parts of this stage: the stride of 10 frames and the window of seven steps from its temporal sweep (Section 3.2.6), and the five kinds of scene cues, which were audited again in the modern stage.

Table (3.3) Quality-oriented profile versus its baseline (YOLOv8n with ByteTrack at default settings) on VisDrone test-dev, historical protocol.

Gain: paired bootstrap difference with 95% interval; for IDS it is the reduction.

Metric	Baseline	Quality	Gain [95% CI]
HOTA	28.430	33.835	+5.4051 [4.1273, 6.9022]
MOTA	19.729	26.948	+7.2195 [5.4362, 9.2934]
IDF1	32.724	41.546	+8.8220 [6.9005, 11.0537]
IDS	1235	1184	51 [−114, 219]
FPS	36.528	38.985	–

3.3.2 Identity-Aware Pareto Profile

Baseline. The baseline is the same YOLOv8n and ByteTrack pipeline at default settings (Section 3.3.1).

Method. The second study kept the same cues, operating levels, temporal settings, and validation data, but changed the question. It maximized MOTA and minimized IDS subject only to FPS≥25, without the IDS gate. A seeded TPE sampler ran 49 completed trials. A trial dominated another when it had no lower MOTA and no higher IDS, with at least one strict improvement, and the feasible Pareto front was built from these relations. The balanced candidate maximized an equal-weight score of normalized MOTA and normalized IDS reduction, with HOTA, IDF1, FPS, and trial index as tie-breaks. The selected profile (archived as Trial 22) has weights w_c=0.1646453, w_t=0.1746265, w_e=0.5076113, w_n=0.1206956, w_b=0.0324213, confidence end points (0.40, 0.40), NMS end points (0.60, 0.35), and thresholds (0.2927136, 0.6661672). On validation it reached MOTA 19.330, HOTA 31.651, IDF1 34.226, and IDS 168 at 51.406 FPS.

Comparison. Table (3.4) compares the profile with the baseline on test-dev. It removes 316 IDS and raises throughput from 36.528 to 46.024 FPS, while HOTA, MOTA, and IDF1 also rise. Against the quality-oriented profile it has 265 fewer IDS [82, 502] and −2.617 HOTA [−4.004, −1.694]. The two profiles therefore answer different questions: one favors overall quality and the other favors identity stability and speed. The detector operating point controls a trade-off between quality, identity stability, and speed, and the objective of the search decides which side is favored.

Table (3.4) Identity-aware profile versus its baseline (YOLOv8n with ByteTrack at default settings) on VisDrone test-dev, historical protocol.

Gain: paired bootstrap difference with 95% interval; for IDS it is the reduction.

Metric	Baseline	Identity	Gain [95% CI]
HOTA	28.430	31.218	+2.788 [0.747, 4.660]
MOTA	19.729	23.792	+4.063 [0.952, 6.886]
IDF1	32.724	37.870	+5.146 [2.028, 8.080]
IDS	1235	919	316 [175, 470]
FPS	36.528	46.024	–

3.3.3 Transfer and Cross-Pipeline Studies

This stage applied the historical controllers outside their development setting: to a second dataset, and to two other detector–tracker pipelines.

UAVDT. The baseline is again YOLOv8n with ByteTrack at default settings (Section 3.3.1). Both frozen profiles were applied to UAVDT [7] without retuning (Table (3.5)). The quality-oriented profile raised HOTA by 4.305 points [2.740, 5.709] and the identity-aware profile by 2.845 points [1.238, 4.348]; their IDS reductions are 237 [104, 377] and 250 [110, 404].

Table (3.5) Historical UAVDT transfer without retuning.

Baseline: YOLOv8n with ByteTrack at default settings.

Metric	Baseline	Quality	Identity
HOTA	24.085	28.390	26.930
MOTA	13.841	17.399	16.118
IDF1	27.887	34.453	32.014
IDS	558	321	308
FPS	65.015	58.353	61.462

U2MOT on VisDrone. The baseline is the published U2MOT tracker [82] with its released YOLOX-X detector, run at its own static setting. Seven VisDrone validation sequences served for development and 17 test-dev sequences for evaluation. The cue weights and transforms of the quality-oriented profile were

kept, the tracker was not modified, and the action boundaries were set to the 33rd and 67th percentiles of the index on the U2MOT validation run, without using ground truth. Table (3.6) (top) gives the comparison: the controlled pipeline has 87 fewer IDS and 1230 fewer false positives, while MOTA is almost unchanged and false negatives are higher.

SparseTrack on MOT17. The baseline is the published SparseTrack tracker [79] with a fixed NMS IoU threshold of 0.75, on the seven MOT17 val-half sequences (2652 frames) [86]. A minimal edge-driven rule switched only the NMS threshold, between 0.70 and 0.80; its threshold came from a leave-one-sequence-out study on the same sequences. Table (3.6) (bottom) gives the comparison: the adaptive rule reduces IDS from 136 to 123, with MOTA +0.0371 points [−0.041, +0.182].

Table (3.6) Cross-pipeline studies.

Top: U2MOT with its YOLOX-X detector at its static setting on VisDrone test-dev. Bottom: SparseTrack with a fixed NMS IoU threshold of 0.75 on MOT17 val-half.

Pipeline	Metric	Baseline	With control
U2MOT	MOTA	53.873	53.767
	IDF1	69.775	69.849
	IDS	1239	1152
	FP	41385	40155
	FN	63241	64801
SparseTrack	MOTA	76.8881	76.9252
	IDF1	81.4943	81.5299
	IDS	136	123

The frozen profiles kept their direction on UAVDT without retuning, and with U2MOT and SparseTrack the control mainly reduced IDS, with small changes in MOTA. Together these studies show that the preferred profile depends on the objective (quality or identity) and on the tracker host. This motivated a modern stage with a stronger detector, explicit calibration data, and two tracker hosts.

3.3.4 Development of the Frozen Controller

The modern stage decided which cues enter the SCI, which operating points each state uses, and where the SCI thresholds lie.

Baseline. The modern baseline is the YOLO11m detector [55], in a public version trained on VisDrone2019-DET [1], run at a fixed operating point of 1536 pixels and NMS IoU 0.70 with no extra score threshold. It feeds ByteTrack [6] during development and both ByteTrack and OATrack [83] in the

final evaluation. A large fixed input is a common choice for small aerial objects when no adaptation is used.

Data and protocol. Eight sequences of the VisDrone2019-MOT training split (3282 frames) were declared as calibration data. They are disjoint from the seven evaluation sequences. The searches used the same four folds, each holding out two calibration sequences. Detector outputs were cached for every resolution and NMS value; a score threshold applied to the cache was checked to reproduce direct inference exactly. The steps are described below in the order in which they were carried out.

Resolution and NMS maps. Two exhaustive grid searches, archived as Stages 1 and 2, searched the state-to-resolution and state-to-NMS maps. At this point the action of a state contained only the input resolution and the NMS value.

Cue audit. The next search, archived as Stage 3A, asked which cues should form the SCI. It searched all 31 non-empty subsets of the five cues and selected {crowd}, which defines the crowd-selected SCI of Eq. (3.5). The other cues were kept as guard and auxiliary cues of the state logic (Section 3.2.3).

Score threshold. A separate sweep then tested the score threshold. At 1088 pixels and NMS 0.45, a fixed score threshold of 0.40 raised pooled calibration HOTA by +0.881 over 0.01 and improved seven of the eight sequences. The score threshold was therefore added as the third action dimension.

Final joint search. The final search then selected the thresholds and the three-part actions together. It used 60 TPE trials (seed 43) and searched $\tau_m \in [0.10, 0.50]$, $\tau_h \in [\tau_m+0.05, \min(\tau_m+0.40, 0.90)]$, and for each state a resolution in {1088, 1280, 1536}, an NMS value in {0.45, 0.60, 0.70}, and a score threshold in {0.10, 0.25, 0.40}. The objective was the cross-validated mean change in HOTA against the baseline, with ByteTrack as host. A candidate was eligible only if it really adapted within the video rather than acting as one fixed operating point: it had to realize at least two distinct actions, each on at least 10% of calibration frames, and switch within sequences. The frozen policy is the eligible candidate whose three states use three different actions that together vary all three parts of the operating point (Table (3.1)). On calibration data it used LOW, MEDIUM, and HIGH on 7.6%, 10.1%, and 82.3% of frames, with 0.607 state changes per 100 frames on average.

Comparison. Table (3.7) compares the frozen policy with the baseline on each calibration fold. All four folds are positive, with a CV mean of +2.447 HOTA. After these steps the thresholds, cue subset, and action map were frozen. The final comparison on the evaluation sequences is given in Chapter 4.

Table (3.7) Frozen AC-MOT policy versus the modern baseline (YOLO11m at 1536 pixels and NMS 0.70, ByteTrack) on the four calibration folds: change in HOTA.

	Fold 1	Fold 2	Fold 3	Fold 4	Mean
ΔHOTA	+0.592	+4.164	+2.029	+3.004	+2.447

3.4 Self-Calibrating Control Layer (SCL)

AC-MOT adapts the detector to the scene, but it assumes that the detector's scores keep the scale for which the tracker was tuned. The second controller addresses this assumption. SCL is a small, rule-based layer that runs once per frame between the detector and the tracker (Fig. (3.2)). It groups the scores of recent frames (Section 3.4.3), decides whether they are clean or noisy (Section 3.4.4), acts on this decision, and can pass a weak candidate that clearly continues a track (Section 3.4.5). The detector and the tracker are never changed. The tracker that SCL serves is called the *host*; it gives SCL four of its published settings, the host contract (Section 3.4.6).

3.4.1 Overview

In short, a detection travels through SCL as follows. Its score is turned into a logit and compared with two splits computed from the scores of the last 10 frames: below the first split it is background, between the splits uncertain, and above the second confident. If most recent foreground scores are confident, the stream is clean and the detection reaches the tracker with its own score. Otherwise the stream is noisy: a background detection is dropped, and the others get new scores that tell the tracker whether they may start a track or only continue one. A weak detection that lies on a track of the previous frame can still be passed on. The tracker then does its own matching as usual. Its score enters the statistics only after the decision for its frame.

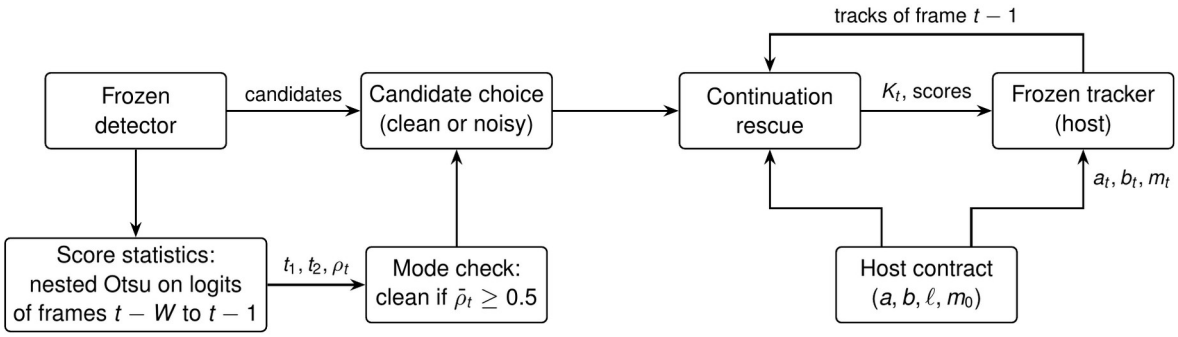


Fig. (3.2) SCL between a frozen detector and a frozen tracker (the host).

Boxes are processing steps and arrows show what each step passes on. The score statistics use earlier frames only. The host contract holds the tracker's published settings. The tracker receives the chosen candidates K_t, their scores, and its thresholds a_t, b_t and tolerance m_t for the frame.

3.4.2 Inputs and Outputs

In each frame t, the detector gives candidates $D_t=\{(\mathbf{x}_i, s_i, c_i)\}$, each with a box $\mathbf{x}_i$, a score $s_i \in (0, 1)$, and a class c_i. The layer works on the logit

$$z_i = \log \frac{s_i}{1-s_i}, \tag{3.13}$$

the log-odds transformation of the confidence score. It converts a score between 0 and 1 into a value on an unrestricted scale while preserving the ordering of scores: a score of 0.5 gives $z_i=0$, 0.9 gives about 2.2, and 0.1 about −2.2. The logistic function $\sigma(z)=1/(1+e^{-z})$ turns a logit back into a score. The layer uses logits for two reasons. Detector scores crowd near 0 and 1, where small differences in score are large differences in confidence; the logit spreads these regions out, so the histogram splits of Section 3.4.3 separate the groups more clearly. And a common change of a detector's scores, a change of temperature, divides all logits by the same number, a simple stretch that the splits follow (Section 3.4.3).

The host gives its contract $h=(a, b, \ell, m_0)$: the score a needed to match a track, the score b needed to start one, the lowest score ℓ it uses, and its matching tolerance m_0, one minus the smallest IoU of two boxes that counts as a match. A single-stage tracker has $\ell=\min(a, b)$, a two-stage tracker $\ell<\min(a, b)$. In every frame the layer returns

$$\left(K_t, \tilde{s}, a_t, b_t, m_t\right) = L\left(Z_{<t}, D_t, T_{t-1}, g_t; h\right): \tag{3.14}$$

the candidates K_t for the tracker, their scores, and its thresholds and tolerance, from the logits $Z_{<t}$ of earlier frames, the tracks T_{t-1}, and a camera-motion measure g_t; no future frame is used.

3.4.3 Self-Calibration from Detector Scores

The layer collects the logits Z_t^W of the candidates of the last W=10 frames and splits them twice with Otsu's method [17] (Eq. (2.3)). The first split, t_1, separates background (low logits) from foreground; the second, t_2, found on the foreground only, divides it into uncertain and confident candidates. The share of confident candidates in the foreground,

$$\rho_t = \frac{\left|\{ z \in Z_t^W : z \geq t_2 \}\right|}{\left|\{ z \in Z_t^W : z \geq t_1 \}\right|}, \tag{3.15}$$

describes the shape of the score distribution and needs no labels: if 30 of 40 foreground candidates are confident, ρ_t=0.75. Otsu's method is used because it picks a split from the data alone, without labels or tuning, and applying it twice gives the three groups that one threshold cannot.

For example, for the scores 0.02, 0.03, 0.05, 0.4, 0.5, 0.9, and 0.95 (logits −3.9, −3.5, −2.9, −0.4, 0, 2.2, and 2.9), the first split separates the three lowest scores as background, and the second divides the other four into 0.4 and 0.5 (uncertain) and 0.9 and 0.95 (confident), so ρ_t=2/4=0.5. Because the splits are made on logits, they move with the scores when the whole logit scale is stretched or shifted, and each group keeps the same candidates. Figure (3.3) shows the groups for a clean and a noisy stream. When a detector emits very few low scores, the first split falls inside the objects; since clutter usually gives more candidates than the objects do, a window with fewer candidates below t_1 than above it counts as clean, ρ_t=1 (background check).

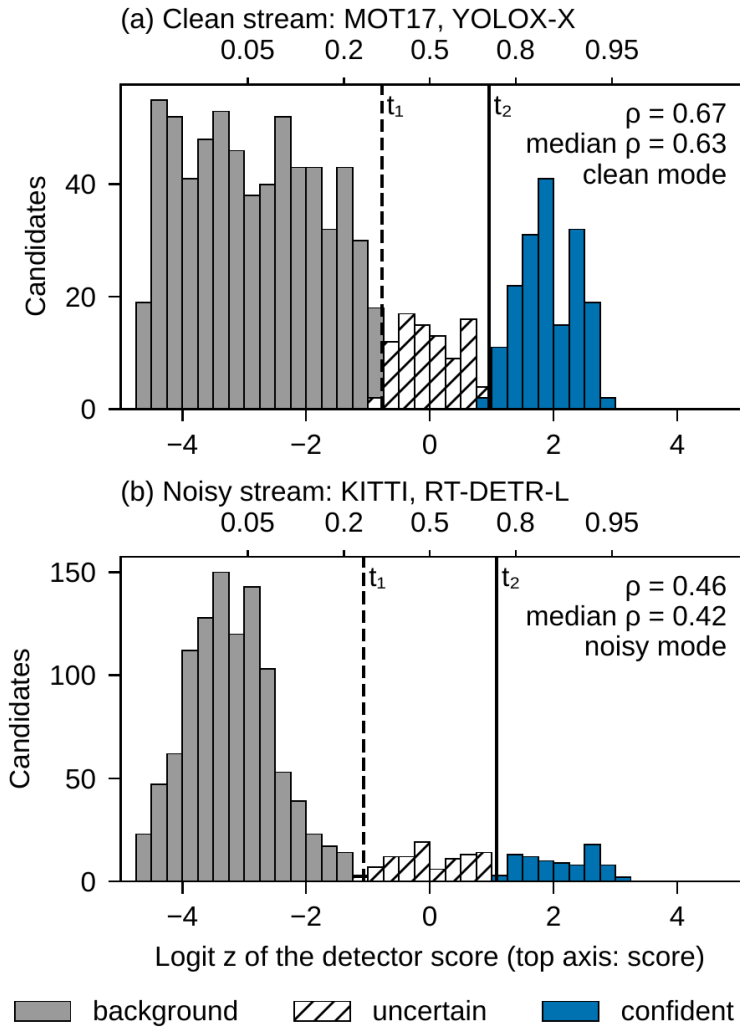


Fig. (3.3) What the layer sees: the logits of the candidates of the last 10 frames, just before its decision for frame 101, with the Otsu splits t_1 (dashed line) and t_2 (solid line).

Plain light bars: background; hatched bars: uncertain; plain dark bars: confident. The top axis gives the matching detector score. (a) MOT17-02 with YOLOX-X: most foreground candidates are confident, and the stream is clean. (b) KITTI 0001 with RT-DETR-L: many are uncertain, and the stream is noisy. Values of the frozen layer in the runs of Chapter 4.

3.4.4 Clean and Noisy Modes

Clean and noisy describe the distribution of the scores, not whether a single detection is correct. The stream is clean at frame t if the median of all ρ values so far is at least one half, a majority rule, and noisy otherwise; before the first valid split it is cold. Because the median covers the whole stream, a short drop in confidence does not change the mode, and during development this whole-stream rule removed mode changes in the middle of streams that had lowered HOTA (Section 3.5.2). The splits themselves use only the last W frames, so the groups, and in the clean mode the threshold cap of Eq. (3.16), follow a change of the score scale within W frames even while the mode stays the same.

Cold and clean modes. The tracker keeps its own thresholds and scores, and only cross-class duplicates are removed. In the clean mode, a threshold so high that it would reject the confident group is lowered to the score $\sigma(t_2)$ at which that group begins:

$$a_t = min\left(a, \sigma\left(t_2\right)\right), b_t = min\left(b, \sigma\left(t_2\right)\right). \tag{3.16}$$

The clean mode is thus a minimal intervention rather than a plain pass-through: the scores are kept, and the output equals that of the host alone whenever the confident group lies above the host's thresholds and no rescue applies, as for BoostTrack, ByteTrack at floor 0.1, and Hybrid-SORT on MOT17 (Chapter 4). When the scores are halved, for example, no score is above 0.5, and ByteTrack in its official setting (a=0.6, b=0.7) never starts a track; the splits move with the halved scores, so Eq. (3.16) lets the confident candidates reach it again.

Noisy mode. Confident candidates ($z \geq t_2$) may start and continue tracks, uncertain ones ($t_1 \leq z < t_2$) may only continue tracks, and background candidates are dropped. Each score is replaced by its mid-rank $u_i \in (0, 1)$ among the scores of a remap memory, which stores the scores kept in the first frame and in every 10th frame, the last 20 of them (tied scores share their average rank, and before the first sample the current frame is used),

$$\tilde{s}_i = \begin{cases} 0.5 + 0.5\,u_i, & z_i \geq t_2, \\ 0.1 + 0.4\,u_i, & t_1 \leq z_i < t_2, \end{cases} \tag{3.17}$$

and both thresholds are set to $a_t = b_t = 0.5$, so uncertain candidates can only reach a tracker's low-score step. A weaker candidate that overlaps a stronger one with IoU above one half is removed, unless a different track of the previous frame claims it, which keeps people who stand close together in a crowd.

3.4.5 Track Continuation and Camera Motion

A track can be lost when its object gets low scores for a few frames, above all in single-stage trackers, which have no low-score step. The rescue passes such a candidate, with its score raised just above the lowest threshold of the frame, when it is in the foreground (or, for a single-stage tracker, when there is no background group), overlaps a track of the previous frame with IoU of at least one half, no other usable candidate covers that track, and the tracker could not see it otherwise. It thus gives single-stage trackers the continuation step they lack. The rescue only makes the box visible; the tracker itself still decides, with its own matching, whether and to which track it is assigned.

In noisy frames, the matching tolerance also grows with camera motion. The motion is the shift between consecutive grayscale frames at quarter resolution, found by phase correlation [105], and r_t is its ratio to its median over the last 100 frames (r_t=1 during a warm-up of 5 frames or if this median is zero):

$$m_t = min\left(0.95, 1 - \frac{1 - m_0}{max\left(1, r_t\right)}\right). \tag{3.18}$$

With m_0=0.8 and twice the usual motion, a match needs an IoU of 0.1 instead of 0.2.

3.4.6 Host Contract and Adapters

Table (3.8) lists the contracts; each value is taken from the tracker's published configuration, and none is fitted. For each tracker family, a small adapter writes the layer's candidates, scores, thresholds, and tolerance into the tracker's existing parameters before each update and then calls the tracker's own update. The matching steps, motion model, track management, and any post-processing of the authors run unchanged, so the host stays frozen: its code, weights, and association algorithm do not change, only the values of its thresholds are set for each frame, and the rules are the same for every tracker.

Table (3.8) Host contracts: the published settings of each tracker, as used by the layer: matching (association) threshold *a*, birth threshold *b*, low threshold ℓ, and match tolerance m_0.

For every tracker, the layer's decision is passed on as thresholds, candidates, and scores.

Tracker (setting)	Type	*a*	*b*	ℓ	m_0
ByteTrack (official MOT17)	two-stage	0.6	0.7	0.1	0.8
ByteTrack (library default)	two-stage	0.25	0.25	0.1	0.8
BoT-SORT (library default)	two-stage	0.25	0.25	0.1	0.8
SparseTrack	two-stage	0.6	0.7	0.1	0.85
BoostTrack	two-stage	0.6	0.6	0.1	0.7
Hybrid-SORT	two-stage	0.6	0.6	0.1	0.75
OC-SORT	single-stage	0.6	0.6	0.6	0.7
PD-SORT	single-stage	0.6	0.6	0.6	0.7

3.4.7 Parameter Values

Table (3.9) lists all values of SCL. They were fixed before the external evaluation, are the same for every tracker, detector, and dataset, and none was chosen by searching over a range of values. They come from standard values (the IoU of 0.5 of MOT evaluation and the common thresholds 0.5 and 0.1), from definitions, and from design choices kept fixed in all development stages. Algorithm 3.2 summarizes the steps for one frame.

Table (3.9) Values of SCL and their source.

Value	Setting	Source
Window W	10 frames	design choice
Mode rule	clean if median $\rho \geq 0.5$	majority rule
Background check	$\rho=1$ if fewer below t_1 than above	rule: no background group
Noisy score ranges	[0.5, 1), [0.1, 0.5)	common thresholds
Noisy a_t, b_t	0.5	common threshold
Remap memory	20 frames, one every 10	design choice
Duplicate, rescue IoU	0.5	MOT evaluation IoU
Rescued score	lowest threshold + 0.001	just above the threshold
Motion cue	phase correlation, quarter resolution	standard method [105]
Motion history	100 frames, warm-up 5	design choice
Tolerance cap	0.95	minimum overlap 0.05

Input: candidates D_t, tracks $\mathcal{T}_{t-1}$, motion cue g_t, contract (a, b, ℓ, m_0), constants of Table (3.9)
Output: candidates K_t, scores $\tilde{s}$, thresholds a_t, b_t, tolerance m_t

$z_i \leftarrow$ logit of s_i for all i ▷ Eq. (3.13)
if no valid split exists **then** ▷ e.g., first frame
    mode $\leftarrow$ cold; $(a_t, b_t) \leftarrow (a, b)$
else
    $(t_1, t_2) \leftarrow$ nested Otsu on the window ▷ Section 3.4.3
    $\rho_t \leftarrow$ Eq. (3.15) with the background check
    mode $\leftarrow$ clean if median$(\rho) \geq 0.5$, else noisy
end if
if mode is cold or clean **then** ▷ Section 3.4.4
    remove cross-class duplicates; keep raw scores
    if clean **then** set (a_t, b_t) by Eq. (3.16)
    end if
else ▷ noisy mode
    remove duplicates with track context
    drop $z < t_1$; remap scores by Eq. (3.17)
    $a_t \leftarrow 0.5$; $b_t \leftarrow 0.5$
end if
rescue continuations (not cold) ▷ Section 3.4.5
$m_t \leftarrow$ Eq. (3.18) if noisy, else m_0
pass K_t, $\tilde{s}$, a_t, b_t, m_t to the tracker
add the logits left after duplicate removal to the window
add g_t to the motion history
store the tracks of frame t after the tracker update

Algorithm 3.2. SCL for frame *t*. The window holds the logits of frames *t*−*W* to *t*−1, so the decision for frame *t* uses no future frame.

3.5 Development of the SCL

SCL was developed in three stages on development data: MOT17 with four trackers, KITTI, and the score transforms of Section 4.2.1. The stages are a starting point, design steps that each removed one cause found before them, and a freeze before any external test.

3.5.1 Starting Point: Always-On Banding

The earlier design, always-on banding (AOB), computed the same nested Otsu groups but treated every stream as noisy: in every frame it applied the groups, removed every weaker candidate that overlapped a stronger one with IoU above one half, and rescored all candidates by rank. Table (3.10) compares AOB and SCL with strong published trackers on MOT17. AOB lowered the HOTA of SparseTrack by 4.15 points and of BoostTrack by 5.89 points online and 5.37 points with its gap-filling post-processing (GBI), for two reasons: removing overlapping boxes deleted people hidden behind others in crowds, and its thresholds were stricter than the trackers' own. With the same trackers, SCL changed HOTA by +0.06 for SparseTrack and left both BoostTrack versions identical to the host. A layer that acts in every frame therefore cannot be placed safely in front of an unknown tracker, and the next steps let the layer decide from the scores whether to act at all.

Table (3.10) Baseline comparison of the development stage on MOT17 val-half (published YOLOX-X detections).

Host: the tracker alone, without any layer (our run of the published tracker). Δ: tracker with the earlier design AOB or with SCL minus host, with 95% paired-bootstrap interval (10,000 resamples, seed 42). Bold intervals exclude zero. IDS: identity switches, a count (lower is better).

Metric	Host	Δ AOB	Δ SCL
SparseTrack [79]			
HOTA	68.88	**−4.15 [−5.54, −1.66]**	+0.06 [−0.01, +0.25]
MOTA	77.85	**−6.14 [−8.10, −2.74]**	+0.08 [−0.03, +0.30]
IDF1	81.97	**−4.49 [−6.19, −1.96]**	+0.16 [−0.02, +0.70]
IDS	124	−16 [−47, +12]	+7 [0, +19]
BoostTrack, online [80]			
HOTA	68.49	**−5.89 [−7.76, −2.87]**	0 (identical)
MOTA	75.50	**−8.87 [−12.37, −7.14]**	0 (identical)
IDF1	81.41	**−6.22 [−9.33, −2.85]**	0 (identical)
IDS	113	**−38 [−81, −1]**	0 (identical)
BoostTrack, with GBI [80]			
HOTA	71.72	**−5.37 [−7.30, −2.75]**	0 (identical)
MOTA	81.03	**−8.73 [−12.59, −6.50]**	0 (identical)
IDF1	84.16	**−5.65 [−8.96, −2.77]**	0 (identical)
IDS	97	−27 [−73, +11]	0 (identical)

3.5.2 Design Steps

Each step was compared with the step before it (Table (3.11)). The *mode gate* added the mode check, the clean mode that keeps the tracker's scores and thresholds, and the crowd-safe duplicate rule; it restored SparseTrack from 64.72 to 68.93 HOTA and BoostTrack from 62.61 to 68.37. The *whole-stream mode* removed mode changes in the middle of streams and raised ByteTrack from 67.561 to 67.702 and OC-SORT from 65.697 to 66.071. The *continuation rescue* added 0.2 to 0.4 points for OC-SORT. The *background check* completed SCL: ByteTrack and BoostTrack returned to output identical to the host, and OC-SORT rose to 67.04 and 66.91 at the two floors. Four other variants were tested and not kept.

Table (3.11) Design steps from AOB to SCL on development data (HOTA, %), and versions that were tested and rejected.

The floor of ByteTrack and OC-SORT is given in parentheses.

Step	**Change**	**Effect**
AOB emulated	always-noisy bands, overlap deduplication, rank remap	ByteTrack (0.1) 56.41 vs host 67.70; OC-SORT (0.1) 45.72 vs 66.44
+ mode gate	mode statistic, host-anchored clean mode, crowd-safe duplicates	SparseTrack 64.72 to 68.93 (host 68.88); BoostTrack 62.61 to 68.37 (host 68.49)
+ whole-stream mode	mode from all past frames	ByteTrack (0.1) 67.561 to 67.702; OC-SORT (0.1) 65.697 to 66.071
+ continuation rescue	foreground track-consistent rescue	OC-SORT (0.01) 66.405 to 66.619; OC-SORT (0.1) 66.071 to 66.469
+ background check = SCL	no background mode gives no evidence against the host	ByteTrack/BoostTrack (0.1) identical to host; OC-SORT 67.041 (0.01), 66.907 (0.1)
Rejected version		*Evidence*
no candidates at frame 1		−0.3 to −0.5 HOTA on every MOT17 host
rescue below a two-stage host's low stage		FP +700 to +1000, −0.4 to −0.5 HOTA
rank-remapped scores in clean frames		ByteTrack −0.2 / −1.6 HOTA
host-clipped noisy primary band		KITTI YOLOv8n ByteTrack +1.4 HOTA but uncontrolled false tracks on a noisy aerial stream (26.9 vs 14.9 boxes/frame)

3.5.3 Freezing and Testing

The final configuration was frozen before any external test as a version-control tag, protected by a lock file with a cryptographic hash of the layer, its configuration, and the evaluation scripts. Sixty-one automated tests check, among others, that the layer uses no future data and passes clean streams unchanged.

3.6 Summary

This chapter presented the two proposed controllers. AC-MOT adapts the detector operating point to the scene: every ten frames it reads cheap cues of the image and of the tracked boxes, forms a crowd-

based scene complexity index, and selects one of three operating points with a three-state controller whose recovery, hysteresis, and dwell rules keep changes rare. Its thresholds and actions were selected by cross-validation on calibration sequences after earlier development stages had shown that the detector operating point controls a trade-off between quality, identity stability, and speed. SCL adapts the tracker's input to the detector's score distribution: nested Otsu splits of recent logits decide whether the scores are clean or noisy; on clean scores the layer keeps the tracker's scores and, where they fit, its thresholds, on noisy scores it chooses the candidates, rescores them, and sets the thresholds, and a continuation rescue gives single-stage trackers the step they lack. It was developed step by step from an earlier always-on design and frozen before any external test. Both controllers are training-free, causal, and leave the detector and the tracker unchanged. The next chapter evaluates them.

Chapter 4

Experimental Results and Performance Evaluation

4.1 Introduction

This chapter evaluates the two controllers of Chapter 3. The experiments ask, for each controller, whether replacing fixed settings by the frozen controller improves tracking with the same detector and tracker, and whether it keeps this effect on data that were not used to build it. Section 4.2 describes the datasets, the evaluation metrics, the statistical comparison, and the implementation environment. Section 4.3 gives the results of AC-MOT on VisDrone2019-MOT and, without retuning, on UAVDT. Section 4.4 gives the results of SCL on seven published trackers under detector and score changes, on predeclared external trackers, and on a held-out aerial test split. Section 4.5 compares the proposed controllers with existing approaches, and Section 4.6 discusses the results.

4.2 Setup Discussion

4.2.1 Datasets

Four public datasets are used. Table (4.1) lists the splits and the role of each one.

VisDrone2019-MOT. VisDrone2019-MOT is a widely used benchmark for MOT in UAV videos [1]. It contains video sequences recorded by various drone-mounted cameras in 14 cities in China, in urban and rural regions and in both sparse and crowded scenes. The VisDrone benchmark annotates ten object categories, such as pedestrians, cars, buses, and bicycles, with boxes that give the location and identity of each tracked object. Its MOT track uses 96 video clips: 56 for training (24,198 frames), 7 for validation (2,846 frames), 16 for challenge testing (6,322 frames), and 17 for development testing (6,635 frames). Although ten categories are annotated, the official MOT evaluation scores only five of them: pedestrian, car, van, bus, and truck [1]. Many objects are small, their number changes quickly in crowded traffic, the moving camera changes position and viewing angle, and lighting, background, and occlusion vary. These properties make the dataset suitable for studying detector settings. Eight training clips serve as calibration data for AC-MOT (Section 3.3.4), and the seven validation clips serve as its evaluation data.

UAVDT. UAVDT is a benchmark designed for object detection and MOT in UAV videos, with a particular focus on vehicles [7]. It consists of 100 video sequences selected from more than 10 hours of video recorded at urban locations such as squares, arterial streets, toll stations, highways, crossings, and

T-junctions, at 30 frames per second and a resolution of 1080×540 pixels. About 80,000 frames are annotated with 0.84 million boxes of vehicles in three categories: car, truck, and bus. The sequences are recorded under different weather conditions, flying altitudes, and camera viewing angles, which change object size, visibility, and the number of vehicles per frame. The test set contains 20 sequences for the MOT task (16,592 frames) [7]. These 20 sequences are used by both controllers, with all vehicles in one class.

MOT17. MOT17 [85] contains 14 sequences of people filmed with static and moving cameras. As is common, the second half of each of the seven training sequences, called val-half (2,652 frames), is used for evaluation [6], with the published YOLOX-X detections [52] at the two published emission floors, the lowest scores emitted, of 0.01 and 0.1. To study score changes directly, these scores were also changed before the tracker by four transforms that keep their order: cubing (s^3), halving ($0.5\,s$), and logit temperatures T=2 and T=0.5, which divide the logit of each score by T.

KITTI tracking. KITTI tracking [3] has 21 training sequences (8,008 frames) recorded from a car and is scored on cars and pedestrians. Here the trackers run with two other detectors, YOLOv8n [54] and RT-DETR-L [59], and keep thresholds that were set for other detectors.

For SCL, MOT17 with four trackers, KITTI, and the score transforms were development data, looked at while the layer was designed. Two predeclared external trackers, PD-SORT and Hybrid-SORT, were chosen in advance and run once with the frozen configuration on the detections released by their authors on MOT17 val-half. The held-out UAVDT test split was also run once, after a written plan. Each external host was checked against its authors' number: EXACT within 0.2 HOTA points, CLOSE within 1.0.

Table (4.1) Datasets and splits used in the experiments and the role of each one.

Dataset	Split	Controller	Role
VisDrone2019-MOT	training, 8 sequences (3,282 frames)	AC-MOT	calibration (tuning) data
VisDrone2019-MOT	validation, 7 sequences (2,846 frames)	AC-MOT	frozen evaluation
UAVDT	test, 20 sequences (16,592 frames)	AC-MOT	transfer without retuning
MOT17	val-half, 7 sequences (2,652 frames)	SCL	development; predeclared external trackers
KITTI tracking	training, 21 sequences (8,008 frames)	SCL	development under a detector change
UAVDT	test, 20 sequences (16,592 frames)	SCL	held-out evaluation, run once

4.2.2 Evaluation Metrics

Eight commonly used MOT metrics are considered: HOTA, MOTA, IDF1, identity switches (IDS), false positives (FP), false negatives (FN), precision, and recall. These metrics measure different aspects of tracking performance, including detection accuracy, object identity consistency, and the ability to track objects across consecutive frames [10, 86]. They are defined below, starting from the errors counted in each frame and ending with the scores that combine them.

Detection errors. The first metrics count the errors made frame by frame. In each frame t, output boxes are matched one to one to ground-truth boxes with an IoU of at least 0.5. Let GT_t be the number of ground-truth boxes, TP_t the number of matched pairs, FP_t the number of output boxes without a match (false positives), and FN_t the number of ground-truth boxes without a match (false negatives, or missed objects). Over all frames,

$$\mathrm{GT} = \sum_t \mathrm{GT}_t, \mathrm{TP} = \sum_t \mathrm{TP}_t, \tag{4.1}$$

$$\mathrm{FP} = \sum_t \mathrm{FP}_t, \mathrm{FN} = \sum_t \mathrm{FN}_t, \tag{4.2}$$

so that $\mathrm{TP}=\mathrm{GT}-\mathrm{FN}$. From these counts, precision is the fraction of output boxes that are correct, and recall is the fraction of ground-truth boxes that are found:

$$\mathrm{Precision} = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}}, \mathrm{Recall} = \frac{\mathrm{TP}}{\mathrm{GT}}. \tag{4.3}$$

Identity switches. The counts above treat each frame separately, so they do not show whether an object keeps the same track over time. Identity switches measure this. Let $\phi_t(i)$ be the output track matched to ground-truth object i in frame t, and $\phi_{\mathrm{prev}}(i)$ the output track to which object i was last matched before t. An identity switch is counted whenever these two tracks differ:

$$\mathrm{IDS} = \sum_t \sum_i 1\left[\phi_t(i) \neq \phi_{\mathrm{prev}}(i)\right], \tag{4.4}$$

where the inner sum runs over the objects that are matched in frame t and were matched before.

MOTA and IDF1. These two scores summarize the errors above in a single number each [101, 102]. MOTA combines the three error types, and IDF1 measures how long each object keeps the correct identity:

$$\mathrm{MOTA} = 1 - \frac{\mathrm{FN} + \mathrm{FP} + \mathrm{IDS}}{\mathrm{GT}}, \tag{4.5}$$

$$\mathrm{IDF1} = \frac{2\,\mathrm{IDTP}}{2\,\mathrm{IDTP} + \mathrm{IDFP} + \mathrm{IDFN}}. \tag{4.6}$$

For IDF1, each ground-truth track is first matched to at most one output track over the whole video. IDTP, IDFP, and IDFN are then the true-positive, false-positive, and false-negative boxes under this track-level matching. Because MOTA counts every false positive and every missed object, it reacts strongly to how many boxes the detector passes on. IDF1 depends more on whether each object keeps one identity.

HOTA. MOTA depends mostly on detection errors, and IDF1 mostly on identity errors. HOTA balances detection and association in one score [10]. At a localization threshold α, let TP_α, FN_α, and FP_α be the matched pairs, missed ground-truth boxes, and unmatched output boxes. For a matched pair c, $\mathrm{TPA}(c)$ is the set of matched pairs with the same ground-truth identity and the same output identity as c; $\mathrm{FNA}(c)$ is the set of boxes of the same ground-truth object that are matched to another output identity or not matched; and $\mathrm{FPA}(c)$ is the set of boxes of the same output track that are matched to another object or not matched. Then

$$\mathrm{DetA}_\alpha = \frac{\left|\mathrm{TP}_\alpha\right|}{\left|\mathrm{TP}_\alpha\right| + \left|\mathrm{FN}_\alpha\right| + \left|\mathrm{FP}_\alpha\right|}, \tag{4.7}$$

$$A(c) = \frac{\left|\mathrm{TPA}(c)\right|}{\left|\mathrm{TPA}(c)\right| + \left|\mathrm{FNA}(c)\right| + \left|\mathrm{FPA}(c)\right|}, \tag{4.8}$$

$$\mathrm{AssA}_\alpha = \frac{1}{\left|\mathrm{TP}_\alpha\right|} \sum_{c \in \mathrm{TP}_\alpha} A(c), \tag{4.9}$$

$$\mathrm{HOTA} = \frac{1}{19} \sum_{\alpha} \sqrt{\mathrm{DetA}_\alpha\, \mathrm{AssA}_\alpha}, \tag{4.10}$$

where α runs over 0.05, 0.10, …, 0.95; DetA is the detection accuracy and AssA the association accuracy.

Evaluation tools. For AC-MOT on VisDrone, results are scored with a Python port of the VisDrone2019-MOT Task 4b evaluation: ignored regions are removed, the five classes are evaluated separately, CLEAR-MOT and identity measures use IoU 0.5, and counts are summed over classes and sequences; HOTA is added with the same class-aware scheme using TrackEval. For SCL, all metrics are computed with TrackEval (commit 12c8791) and the official ground truth [86]; KITTI HOTA is

averaged over cars and pedestrians. On UAVDT all vehicles form one class and predictions in ignored regions are removed.

Units and direction. HOTA, MOTA, IDF1, precision, and recall are given in percent, and differences in points; IDS, FP, and FN are counts; speed is given in FPS, and runtime in milliseconds (ms) per frame. Higher values are better for HOTA, MOTA, IDF1, precision, recall, and FPS, and lower values are better for IDS, FP, FN, and runtime.

4.2.3 Statistical Comparison

All comparisons are paired: each tracker runs with fixed settings and with the controller on the same sequences and detections, and nothing else changes. To judge whether the difference between two systems is reliable, paired bootstrap resampling is applied at the sequence level [103], which keeps the dependence within each sequence. Each resample draws the evaluation sequences with replacement; the same draw is used for both systems, the pooled metrics are recomputed, and their difference is recorded. The 95% interval is the 2.5–97.5 percentile range, and a difference is called significant when its interval does not include zero. The AC-MOT comparisons use 5000 resamples and the SCL comparisons 10,000 resamples, both with seed 42. Because whole sequences are resampled, the intervals reflect the variation between videos, not only between frames.

4.2.4 Implementation Environment and Toolkit

The controllers and the evaluation are implemented in Python. The image cues use OpenCV (the Canny edge detector, the Laplacian, and phase correlation) and NumPy; the detectors run with PyTorch and the Ultralytics library; the tracking metrics are computed with TrackEval; and the development searches of AC-MOT use Optuna [100]. Table (4.2) summarizes the detectors, trackers, and hardware of each controller.

AC-MOT. All experiments use the YOLO11m detector [55] in a public version trained on VisDrone2019-DET. It runs in half precision, and detections of the classes pedestrian, car, van, truck, and bus are passed on. ByteTrack uses the Ultralytics implementation with its default parameters: high threshold 0.25, low threshold 0.10, new-track threshold 0.25, buffer 30 frames, and match threshold 0.80. OATrack is a re-implementation of the published method [83] in its main configuration, with its published tracker-side confidence gate of 0.40. This gate belongs to the tracker and is never changed by AC-MOT. For each host, two systems are compared. The baseline runs the detector at the fixed operating

point of 1536 pixels and NMS IoU 0.70, with a minimal score threshold of 0.01 (Section 3.2.6). AC-MOT runs the frozen controller of Section 3.2 on the same detector, so the two systems differ only in the detector operating point. Latency is measured on one Tesla T4 GPU over the first 200 frames of sequence uav0000086_00000_v, excluding five warm-up frames. For every frame, the time spent in the controller (image statistics, scene analysis, and state update) is recorded separately from the detector and tracker time.

SCL. Seven published trackers serve as hosts, each with the authors' code and published configuration: ByteTrack [6], BoT-SORT [67], OC-SORT [65], SparseTrack [79], BoostTrack [80], PD-SORT [81], and Hybrid-SORT [78]. ByteTrack is used with the thresholds that its authors used on MOT17 (official setting) and with the defaults of the Ultralytics library, which also provides BoT-SORT (library setting). For each host, the baseline is the host alone (Host in the tables), run with its published configuration and the same detections; the second run adds the frozen SCL and changes nothing else. The runtime of SCL is measured on a 4-vCPU Intel Xeon processor at 2.10 GHz without a GPU, with YOLOv8n and ByteTrack on three KITTI sequences (2,279 timed frames).

Table (4.2) Detectors, trackers, data, and hardware used to evaluate the two controllers.

Item	AC-MOT	SCL
Detectors	YOLO11m (VisDrone2019-DET weights), half precision	YOLOX-X (published MOT17 detections), YOLOv8n, RT-DETR-L
Trackers	ByteTrack (Ultralytics), OATrack	ByteTrack, BoT-SORT, OC-SORT, SparseTrack, BoostTrack, PD-SORT, Hybrid-SORT
Data	VisDrone2019-MOT, UAVDT	MOT17, KITTI tracking, UAVDT
Baseline	fixed operating point (1536, 0.70)	host alone with its published configuration
Bootstrap	5000 resamples, seed 42	10,000 resamples, seed 42
Runtime hardware	Tesla T4 GPU	4-vCPU Intel Xeon at 2.10 GHz, no GPU

4.3 Results of the Scene-Adaptive Controller (AC-MOT)

4.3.1 Frozen VisDrone Evaluation

The VisDrone evaluation uses seven sequences of the VisDrone2019-MOT validation split. They contain 71,830 ground-truth boxes. The cue subset, thresholds, and action map of the frozen controller were selected on the separate calibration sequences only (Section 3.3.4). The bootstrap draws seven sequences with replacement. Table (4.3) lists the aggregate results on the seven VisDrone sequences, and Fig. (4.1) shows HOTA, MOTA, and IDF1.

Table (4.3) Frozen VisDrone evaluation on seven VisDrone2019-MOT sequences (71,830 ground-truth boxes).

Baseline: fixed operating point (1536, 0.70).

	ByteTrack		OATrack	
Metric	**Baseline**	**AC-MOT**	**Baseline**	**AC-MOT**
HOTA	41.586	44.523	46.770	48.095
MOTA	9.876	27.871	28.165	34.483
IDF1	48.569	53.929	57.107	59.364
IDS	1341	1016	773	604
FP	39274	24892	25394	21400
FN	24121	25902	25432	25057
Precision	54.849	64.852	64.628	68.609
Recall	66.419	63.940	64.594	65.116

ByteTrack host. AC-MOT raises HOTA, MOTA, and IDF1 and lowers IDS and FP, while FN rise from 24121 to 25902 and recall falls from 66.419 to 63.940. The detector output becomes more selective: clutter and identity-confusing boxes are removed at a small cost in recall. The very low baseline MOTA shows how many FP the fixed operating point passes to this tracker; because MOTA counts every FP, the removal of more than a third of them is the main source of the MOTA gain.

OATrack host. OATrack starts from a stronger baseline, and its gains are smaller, but every metric of Table (4.3) moves in the direction of improvement, including FN (25432 to 25057) and recall (64.594 to 65.116).

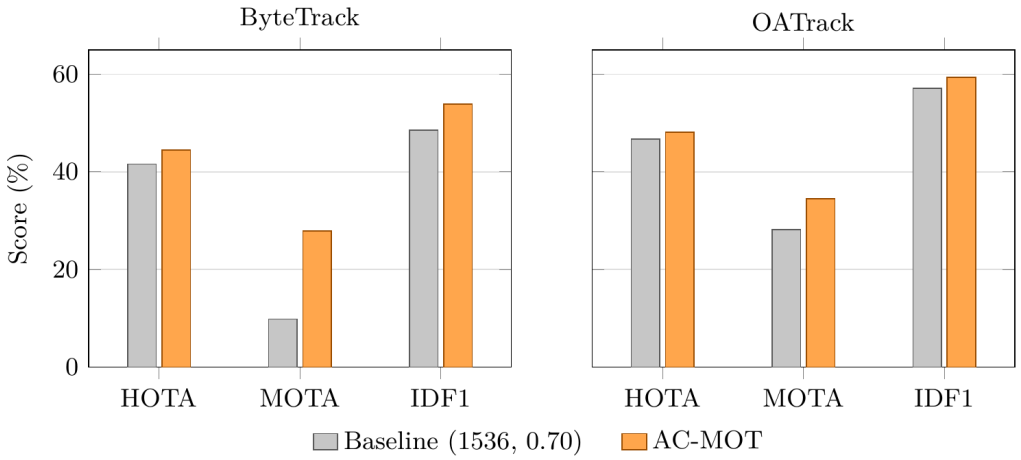


Fig. (4.1) HOTA, MOTA, and IDF1 on the seven VisDrone sequences for both tracker hosts (values in Table (4.3)).

Paired bootstrap. In the VisDrone columns of Table (4.4), the ByteTrack intervals of HOTA, MOTA, IDF1, precision, IDS, and FP lie entirely on the side of improvement, while FN and recall move

in the opposite direction. With OATrack, HOTA, MOTA, IDF1, and precision rise and IDS and FP fall, all with intervals on one side of zero.

Table (4.4) Paired sequence-level bootstrap of AC-MOT: AC-MOT minus baseline, 5000 resamples, seed 42, on VisDrone (7 sequences) and UAVDT (20 sequences).

HOTA, MOTA, IDF1, precision, and recall in points; IDS, FP, and FN in counts. On UAVDT the bootstrap covers the count-based metrics only.

		VisDrone		UAVDT	
Host	**Metric**	**Difference**	**95% interval**	**Difference**	**95% interval**
ByteTrack	HOTA	+2.937	[+2.356, +3.375]	–	
	MOTA	+17.995	[+13.541, +23.282]	+17.601	[+11.134, +26.767]
	IDF1	+5.360	[+4.490, +6.178]	+3.367	[+1.676, +5.013]
	IDS	−325	[−498, −164]	−115	[−211.025, −26.975]
	FP	−14382	[−19176, −9620]	−67539	[−104215.350, −37352.950]
	FN	+1781	[+288, +3594.075]	+7652	[+1542.925, +18359.400]
	Precision	+10.003	[+8.712, +11.895]	–	
	Recall	−2.479	[−4.507, −0.469]	–	
OATrack	HOTA	+1.325	[+0.200, +2.264]	–	
	MOTA	+6.318	[+4.741, +7.879]	+3.267	[+0.607, +5.881]
	IDF1	+2.258	[+0.548, +3.474]	+0.831	[+0.125, +1.271]
	IDS	−169	[−292, −42]	−31	[−101, +39]
	FP	−3994	[−6190, −2494.900]	−14268	[−33958.075, −2188.950]
	FN	−375	[−1462.575, +670]	+3162	[−211.100, +8959.050]
	Precision	+3.981	[+2.957, +5.511]	–	
	Recall	+0.522	[−0.869, +2.111]	–	

4.3.2 Controller Behavior

The controller uses all three states with both hosts. With ByteTrack, LOW, MEDIUM, and HIGH cover 7.4%, 30.1%, and 62.5% of the frames; with OATrack they cover 7.4%, 33.3%, and 59.3%. For most of the video, the detector therefore runs with the HIGH action, the smallest input and a score threshold of 0.40. The two hosts diverge through their different tracked boxes. State changes are rare, between 0.10 and 0.54 per 100 frames depending on the sequence, so the detector input stays stable over long stretches of video.

4.3.3 Frozen UAVDT Transfer

The UAVDT transfer tests whether the frozen controller still helps on a different dataset with none of its values changed. It uses 20 test sequences with 16,592 frames, scored with one vehicle class. Detector outputs were produced at the three frozen operating points (49,776 detector forward passes), with a single

frozen run per host. The paired bootstrap draws 20 sequences with replacement from the stored per-sequence counts and therefore covers the count-based metrics MOTA, IDF1, IDS, FP, and FN.

Table (4.5) Frozen UAVDT transfer of AC-MOT without retuning (20 sequences, 16,592 frames, one vehicle class).

Baseline: fixed operating point (1536, 0.70).

	ByteTrack		OATrack	
Metric	**Baseline**	**AC-MOT**	**Baseline**	**AC-MOT**
HOTA	45.923	47.004	47.728	47.640
MOTA	−1.573	16.028	19.607	22.873
IDF1	58.400	61.767	62.101	62.931
IDS	601	486	517	486
FP	279616	212077	197937	183669
FN	66050	73702	75612	78774
Precision	49.571	55.751	57.270	58.800
Recall	80.625	78.381	77.820	76.893

Table (4.5) and Fig. (4.2) report the transfer to UAVDT without retuning. With ByteTrack, HOTA rises by 1.081 points, MOTA by 17.601, and IDF1 by 3.367; with OATrack, MOTA rises by 3.267 and IDF1 by 0.831, while HOTA is almost unchanged (47.728 and 47.640). With both hosts FP and IDS fall and precision rises, with slightly lower recall, the same pattern as with ByteTrack on VisDrone: the controller changes the detector output in the same way on a dataset that was not used to build it.

In the UAVDT columns of Table (4.4), the ByteTrack intervals of MOTA, IDF1, IDS, and FP lie on the side of improvement and FN moves the other way; with OATrack, MOTA and IDF1 rise and FP fall with intervals on one side of zero, while the IDS and FN intervals include zero.

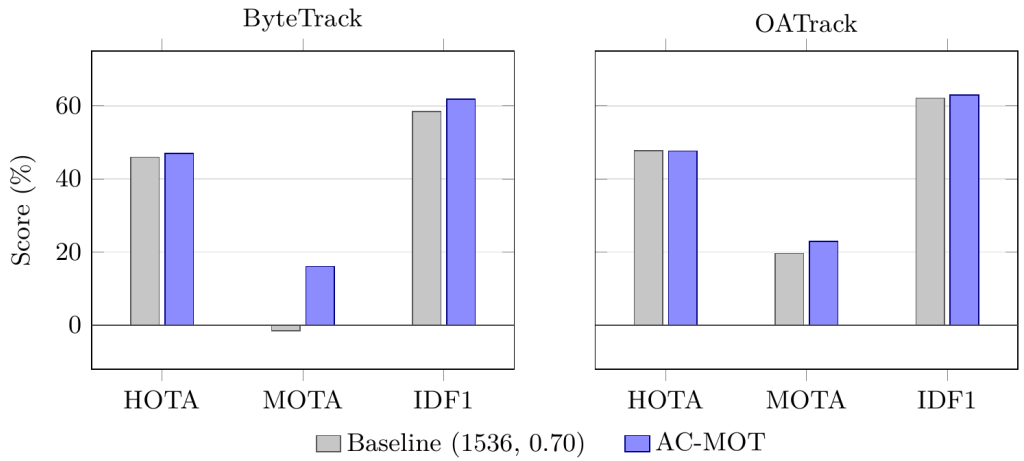


Fig. (4.2) HOTA, MOTA, and IDF1 of the frozen UAVDT transfer of AC-MOT for both tracker hosts (values in Table (4.5)).

The ByteTrack baseline MOTA is negative (−1.573); the solid line marks zero.

4.3.4 Runtime of AC-MOT

AC-MOT needs no training and no access to detector or tracker internals; it changes three arguments of the detector call and analyzes a quarter-scale image every ten frames. On a Tesla T4 the controller takes 4.26 ms per frame with ByteTrack and 4.20 ms with OATrack, which is small next to detector inference and suitable for real-time use.

4.4 Results of the Self-Calibrating Control Layer (SCL)

A tracker that fits its detector should keep its accuracy, and one that does not, because it lacks a continuation step or because the detector or its scores changed, should gain. Figure (4.3) summarizes the HOTA difference for every tested cell.

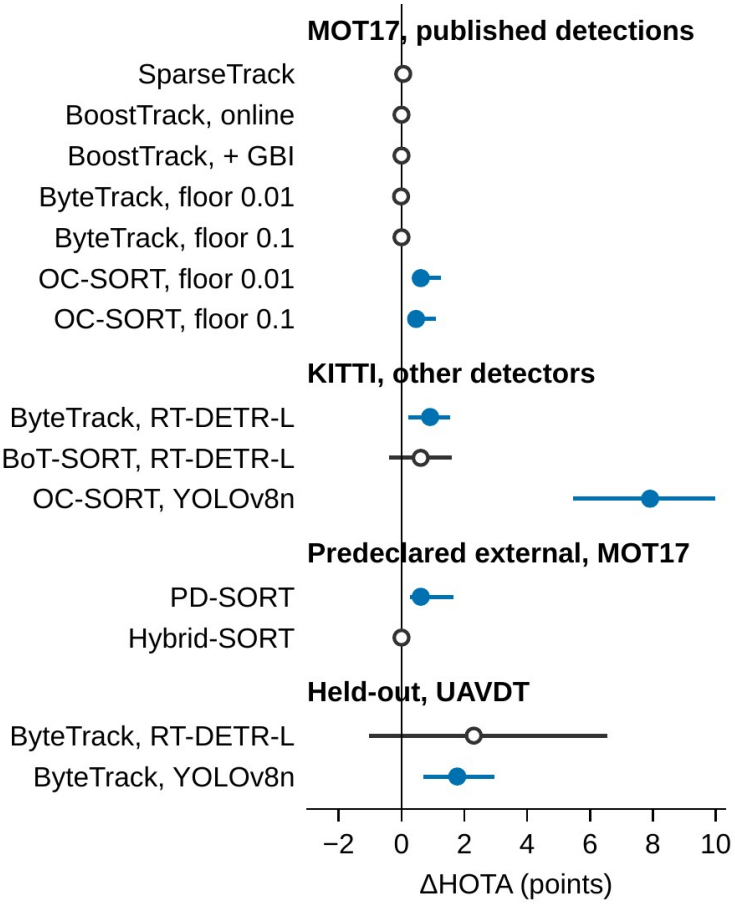


Fig. (4.3) Difference in HOTA, host with SCL minus host alone, in points, for every tested cell, with its 95% paired-bootstrap interval.

Filled circles: interval above zero; open circles: interval contains zero, or identical output.

4.4.1 MOT17 Evaluation Across Trackers

Table (4.6) shows the development trackers on MOT17 val-half. The two-stage trackers with well-matched detections stayed at the accuracy of their hosts: BoostTrack, online and with GBI, and ByteTrack at floor 0.1 gave exactly the same output, and ByteTrack at floor 0.01 and SparseTrack stayed within 0.06 HOTA points, the self-limiting clean mode. The single-stage OC-SORT improved at both floors (HOTA +0.61 and +0.46, MOTA +1.27 and +1.26), with fewer identity switches and higher HOTA on 7 of 7 sequences, through the continuation rescue.

Table (4.6) Baseline comparison of the development trackers on MOT17 val-half (published YOLOX-X detections).

Paper: number reported by the tracker's authors. Host: the tracker alone, without SCL (our run of the published tracker). Δ: host with SCL minus host, with 95% paired-bootstrap interval (10,000 resamples, seed 42); bold intervals exclude zero. The floor is the lowest detector score emitted. IDS: identity switches, a count (lower is better); –: not reported by the authors.

Metric	Paper	Host	Δ [95% CI]
SparseTrack [79]			
HOTA	69.2	68.88	+0.06 [−0.01, +0.25]
MOTA	76.8	77.85	+0.08 [−0.03, +0.30]
IDF1	81.4	81.97	+0.16 [−0.02, +0.70]
IDS	–	124	+7 [0, +19]
BoostTrack, online [80]			
HOTA	68.37	68.49	0 (identical)
MOTA	75.56	75.50	0 (identical)
IDF1	81.35	81.41	0 (identical)
IDS	118	113	0 (identical)
BoostTrack, + GBI [80]			
HOTA	71.33	71.72	0 (identical)
MOTA	80.55	81.03	0 (identical)
IDF1	83.84	84.16	0 (identical)
IDS	106	97	0 (identical)
ByteTrack, floor 0.01 [6]			
HOTA	–	67.70	−0.014 [−0.058, +0.009]
MOTA	76.6	77.60	+0.06 [−0.15, +0.34]
IDF1	79.3	79.47	−0.031 [−0.135, +0.035]
IDS	–	214	+4 [−2, +13]
ByteTrack, floor 0.1 [6]			
HOTA	–	67.70	0 (identical)
MOTA	76.6	77.60	0 (identical)
IDF1	79.3	79.47	0 (identical)
IDS	–	214	0 (identical)
OC-SORT, floor 0.01 [65]			
HOTA	66.5	66.43	**+0.61 [+0.36, +1.24]**
MOTA	74.9	74.67	**+1.27 [+0.17, +3.01]**
IDF1	77.7	78.05	+0.66 [−0.25, +1.60]
IDS	–	211	**−8 [−15, −2]**
OC-SORT, floor 0.1 [65]			
HOTA	66.5	66.44	**+0.46 [+0.27, +1.09]**
MOTA	74.9	74.67	**+1.26 [+0.37, +3.34]**
IDF1	77.7	78.05	**+0.287 [+0.003, +0.838]**
IDS	–	213	−14 [−38, +6]

4.4.2 KITTI Evaluation with Other Detectors

Table (4.7) shows what happens when the detector is replaced. With the noisy RT-DETR-L, ByteTrack gained 5.98 MOTA, 3.87 IDF1, and 0.91 HOTA points, and BoT-SORT 7.86 MOTA and 3.10 IDF1 points, with 297 and 226 fewer identity switches. With YOLOv8n, OC-SORT gained 7.91 HOTA points ([+5.47, +9.98]), 9.40 MOTA points, and 9.60 IDF1 points; it followed many more objects, as its missed objects (FN) fell by 5,310, and its identity switches rose with them by 79. Figure (4.4) follows this

cell over three frames: each object keeps its ID, and at frame 102 OC-SORT alone tracked 6 of the 10 labelled objects and with SCL 9.

Table (4.7) Baseline comparison under a detector change on KITTI tracking training (official HOTA, car and pedestrian averaged).

Host: the tracker alone, without SCL, with the same detections. Δ: host with SCL minus host, with 95% paired-bootstrap interval; bold intervals exclude zero. IDS: identity switches, a count (lower is better). BoT-SORT: KITTI sequence 0020 is left out of both runs, because the BoT-SORT code stopped with a numerical error in its Kalman update on it; the other 20 sequences are used.

Metric	**Host**	**Δ [95% CI]**
ByteTrack [6], *RT-DETR-L*		
HOTA	50.73	**+0.91 [+0.22, +1.55]**
MOTA	47.27	**+5.98 [+1.16, +9.94]**
IDF1	64.98	**+3.87 [+1.67, +5.06]**
IDS	686	**−297 [−530, −122]**
BoT-SORT [67], *RT-DETR-L* (20 sequences)		
HOTA	53.91	+0.61 [−0.39, +1.61]
MOTA	48.25	**+7.86 [+2.21, +13.44]**
IDF1	67.74	**+3.10 [+1.00, +4.57]**
IDS	476	**−226 [−369, −120]**
OC-SORT [65], *YOLOv8n*		
HOTA	36.27	**+7.91 [+5.47, +9.98]**
MOTA	33.99	**+9.40 [+1.10, +12.30]**
IDF1	50.46	**+9.60 [+5.61, +12.16]**
IDS	91	**+79 [+43, +125]**

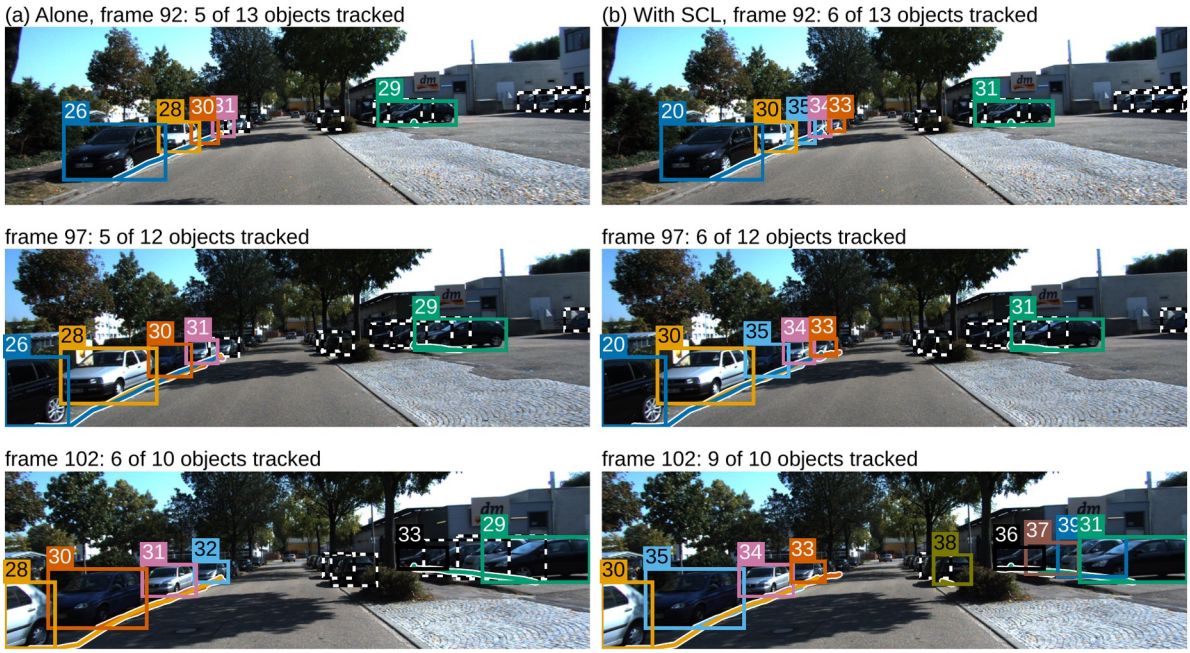


Fig. (4.4) Tracking on KITTI sequence 0001 [3] with OC-SORT and YOLOv8n, at frames 92, 97, and 102 (rows): (a) the tracker alone, (b) with SCL.

Each output track keeps one ID and one colour over the frames; the line of the same colour is its path over the previous 20 frames (bottom centre of its box). Dashed boxes: labelled objects that no track covers. IDs are numbered separately in each run. The sequence has the median HOTA gain of the 21 sequences, and frame 102 is its first frame with the largest gain in tracked objects.

4.4.3 Evaluation under Shifted Detector Scores

Table (4.8) shows the trackers under the four score transforms. Halving the scores reduced ByteTrack (official setting) and OC-SORT to 0 HOTA; with the layer they reached 66.17 and 66.72. Cubing reduced them to 60.92 and 58.34, and the layer restored them to 65.81 and 66.60; BoostTrack rose from 61.33 to 64.84. With T=2, the layer raised ByteTrack and OC-SORT by 1.11 and 1.24 points. The library setting of ByteTrack, whose low thresholds already accept the changed scores, stayed within 0.14 points. Over the 14 conditions, the layer raised HOTA significantly in 7 and lowered it significantly in none.

Table (4.8) Baseline comparison under shifted detector scores on MOT17 val-half.

The detector scores are transformed before the tracker, whose published thresholds stay fixed; T is a logit temperature, $\sigma(z/T)$. Host: the tracker alone, without SCL. Δ: with SCL minus host, with 95% paired-bootstrap interval (10,000 resamples, seed 42); bold intervals exclude zero. Emission floor 0.01 (BoostTrack 0.1).

	HOTA (%)		
Transform	**Host**	**+ SCL**	**Δ [95% CI]**
ByteTrack, official setting [6]			
s^3	60.92	65.81	**+4.89 [+2.60, +13.69]**

	HOTA (%)		
Transform	**Host**	**+ SCL**	**Δ [95% CI]**
0.5 *s*	0.00	66.17	**+66.17 [+49.32, +75.40]**
T=2	65.84	66.95	**+1.11 [+0.35, +3.87]**
T=0.5	67.30	67.29	−0.009 [−0.051, +0.005]
ByteTrack, library setting [6]			
s^3	66.31	66.31	−0.006 [−0.384, +0.383]
0.5 *s*	66.61	66.48	−0.14 [−0.66, +0.14]
T=2	63.97	63.97	0 (identical)
T=0.5	66.53	66.52	−0.006 [−0.029, +0.001]
OC-SORT [65]			
s^3	58.34	66.60	**+8.26 [+5.86, +16.81]**
0.5 *s*	0.00	66.72	**+66.72 [+50.32, +75.56]**
T=2	65.99	67.22	**+1.24 [+0.52, +3.42]**
T=0.5	66.61	67.06	+0.45 [−0.01, +0.97]
BoostTrack, online [80]			
s^3	61.33	64.84	**+3.51 [+0.85, +13.28]**
T=2	67.59	67.59	0 (identical)

4.4.4 Frozen Evaluation on External Trackers

Table (4.9) gives the two predeclared external systems. PD-SORT [81] was reproduced exactly. With SCL, its HOTA rose by 0.61 points ([+0.27, +1.65]), its MOTA by 1.13 ([+0.27, +3.03]), and its IDF1 by 0.82 ([+0.43, +1.94]), with higher HOTA on 7 of 7 sequences; like OC-SORT, it is a single-stage tracker, but it was not seen during design. Hybrid-SORT [78], a two-stage tracker, gave identical output, as predicted before the run.

Table (4.9) Baseline comparison on the predeclared external systems (MOT17 val-half, authors' released detections), each evaluated once with the frozen SCL.

Paper: number reported by the authors (for PD-SORT, their released output). Host: the tracker alone, without SCL (our run of the published tracker). Reproduction class against the authors' number (EXACT: $|\Delta\text{HOTA}| \leq 0.2$; CLOSE: ≤1.0 with a documented, untuned cause). Δ: host with SCL minus host, with 95% paired-bootstrap interval; bold intervals exclude zero. IDS: identity switches, a count (lower is better); –: not reported.

Metric	**Paper**	**Host**	**Δ [95% CI]**
PD-SORT [81], *EXACT*			
HOTA	68.01	68.01	**+0.61 [+0.27, +1.65]**
MOTA	75.19	75.19	**+1.13 [+0.27, +3.03]**
IDF1	81.03	81.03	**+0.82 [+0.43, +1.94]**
IDS	–	129	+2 [−9, +13]
Hybrid-SORT [78], *CLOSE*			
HOTA	67.1	66.70	0 (identical)
MOTA	75.8	75.52	0 (identical)
IDF1	78.0	77.56	0 (identical)
IDS	–	259	0 (identical)

4.4.5 Frozen UAVDT Transfer

On the held-out UAVDT test split (Table (4.10)), SCL raised the HOTA of the YOLOv8n stream by 1.77 points ([+0.70, +2.94]) and its IDF1 by 3.97 points, and cut identity switches by 45%. With RT-DETR-L, whose host produced many false positives (MOTA 3.29), SCL raised MOTA by 24.70 points ([+10.06, +51.08]) and IDF1 by 5.74 points, and cut false positives by 58% and identity switches by 56%. No HOTA decrease was detected with either detector.

Table (4.10) Baseline comparison on the held-out UAVDT test split (20 aerial sequences, not used during design), evaluated once with the frozen SCL.

Host: ByteTrack alone, without SCL, with its published setting. Δ: host with SCL minus host, with 95% paired-bootstrap interval; bold intervals exclude zero. IDS: identity switches, a count (lower is better).

Metric	Host	Δ [95% CI]
ByteTrack, RT-DETR-L [6, 59]		
HOTA	40.07	+2.30 [−1.05, +6.55]
MOTA	3.29	**+24.70 [+10.06, +51.08]**
IDF1	48.59	**+5.74 [+0.13, +13.11]**
IDS	1041	**−586 [−951, −248]**
ByteTrack, YOLOv8n [6, 54]		
HOTA	29.05	**+1.77 [+0.70, +2.94]**
MOTA	16.27	+1.02 [−1.06, +2.86]
IDF1	34.52	**+3.97 [+1.88, +6.17]**
IDS	719	**−326 [−691, −73]**

4.4.6 Runtime of SCL

SCL needs no training, no GPU, and no access to the inside of the detector or the tracker. On the CPU of Table (4.2) it takes 2.95 ms per frame, small next to the 42 ms of the detector, and the tracker itself ran faster with it (1.43 against 2.04 ms per frame); the end-to-end time rose by only 4.5%. A new tracker is added through its four published contract values and an adapter, without changes to the rules.

4.5 Comparison with Existing Approaches

4.5.1 Comparison with Adaptive-Threshold Baselines

Two simple baselines set the tracker's matching and birth thresholds in every frame to the first or second Otsu split of the logits of the last 10 frames, t_1 or t_2, and pass the scores unchanged. On KITTI (Table (4.11)), SCL reached the highest HOTA in all three cells, 2.2 to 8.6 points above the baselines, with intervals against the t_2 baseline above zero in every cell, so adaptive thresholds alone do not explain

the gains. With OC-SORT, SCL also had fewer identity switches than both baselines (170 against 1,545 and 250 with YOLOv8n; 308 against 1,975 and 492 under halved scores).

Table (4.11) HOTA (%) of SCL and of two simple adaptive-threshold baselines on KITTI tracking training (official HOTA, car and pedestrian averaged).

Host: the tracker alone, without any layer. t_1, t_2: the adaptive baselines, in which the tracker's matching and birth thresholds are set in every frame to the first or second Otsu split of the logits of the last 10 frames, and the scores are passed unchanged. Halved: halved detector scores. Δ: SCL minus the t_2 baseline, with 95% paired-bootstrap interval; bold intervals exclude zero.

Cell	Host	t_1	t_2	SCL	Δ [95% CI]
OC-SORT, YOLOv8n	36.27	39.97	41.09	44.17	**+3.09 [+2.00, +4.03]**
ByteTrack, RT-DETR-L, halved	49.50	49.58	49.50	51.78	**+2.29 [+0.28, +4.96]**
OC-SORT, YOLOv8n, halved	0.00	37.68	43.56	46.30	**+2.74 [+1.32, +3.34]**

4.5.2 Comparison with ACT

ACT [16] was run with its published parameters, without retuning, on the same detections (Table (4.12)). SCL reached higher HOTA than ACT in every row, significantly under halved scores (+2.01 and +10.06 points) and with YOLOv8n on UAVDT (+1.82), and also in the MOT17 setting for which ACT was designed. With RT-DETR-L on UAVDT, SCL reached a MOTA of 27.99 against 5.34 (+22.65, [+9.03, +47.21]), because the fixed score levels of ACT removed few of the host's false positives (178,219 against 78,975). SCL also had fewer identity switches than ACT on MOT17 and UAVDT (up to 418 fewer).

Table (4.12) HOTA of ACT and of SCL on the same detections, and the differences SCL minus ACT in HOTA and IDF1, with 95% paired-bootstrap interval; bold intervals exclude zero.

ACT: published parameters, no tuning. KITTI: official HOTA, car and pedestrian averaged; MOT17: val-half, ACT's own setting; UAVDT: test split, one vehicle class, ignored regions removed.

Data	Detector	Tracker	ACT	SCL	ΔHOTA	ΔIDF1
KITTI, halved scores	RT-DETR-L	ByteTrack	49.77	51.78	**+2.01 [+0.01, +4.76]**	+1.70 [−1.13, +6.80]
KITTI, halved scores	YOLOv8n	OC-SORT	36.24	46.30	**+10.06 [+6.88, +12.01]**	**+12.03 [+6.85, +14.69]**
MOT17, floor 0.01	YOLOX-X	ByteTrack (official)	67.46	67.68	+0.22 [−0.43, +0.88]	+0.57 [−0.68, +2.12]
MOT17, floor 0.1	YOLOX-X	ByteTrack (official)	67.46	67.70	+0.23 [−0.40, +0.90]	+0.60 [−0.55, +2.10]
UAVDT test	YOLOv8n	ByteTrack	29.01	30.83	**+1.82 [+0.65, +3.00]**	**+4.00 [+1.85, +6.22]**
UAVDT test	RT-DETR-L	ByteTrack	40.14	42.37	+2.23 [−0.97, +6.32]	+5.35 [−0.03, +12.47]

4.5.3 Contribution of Each Part of SCL

Removing one part at a time on KITTI shows where the gains come from: without the rescue, HOTA fell by up to 3.35 points; without the mode decision, by up to 2.44; and without the new scores of the noisy mode, by up to 1.67. Each part therefore contributes to the result, and the rescue contributes most.

4.5.4 Comparison of Properties

Table (4.13) compares the proposed controllers with the closest adaptive methods by what they adapt, where they act, and for which task they are designed. AC-MOT is the only method in the table that adapts the confidence threshold, the NMS threshold, and the input resolution of the detector together for tracking, with the tracker unchanged. SCL is the only one that serves several unchanged trackers through their published thresholds and decides from the score distribution of past frames whether to act at all.

Table (4.13) Adaptive methods closest to the proposed controllers.

Method	Adapted	Where	Task
AdaScale [11]	input scale	detector	video detection
ApproxDet [12]	detection configuration	detector	video detection
Chameleon [13]	pipeline configuration	analytics pipeline	video analytics
Adaptive ByteTrack [14]	confidence threshold	inside ByteTrack	MOT
AdapTrack [15]	matching thresholds	inside the tracker	MOT
ACT [16]	high and low thresholds from the current scores	tracker thresholds	MOT
AC-MOT (this thesis)	confidence, NMS, resolution	detector; tracker unchanged	aerial MOT
SCL (this thesis)	candidates, scores, thresholds, matching tolerance	between detector and tracker; both unchanged	MOT, several trackers

4.6 Discussion

The operating point is part of the MOT system. The same frozen detector and trackers give very different MOT results at different operating points: moving away from the fixed 1536/0.70 point raises MOTA by 17.995 points with ByteTrack and by 6.318 points with OATrack on VisDrone. On both aerial datasets FP and IDS drop sharply and precision rises, while FN rise a little, so MOTA, which counts every FP, reacts most strongly. The detector operating point is therefore a first-order variable of aerial MOT and deserves the same attention as the association method.

The score scale is part of the MOT system. Halving the scores reduced ByteTrack and OC-SORT from 67.70 and 66.43 HOTA to 0, and with RT-DETR-L on UAVDT ByteTrack reached a MOTA of only 3.29. The fixed thresholds of a tracker hold only for the detector they were tuned for, so the score scale deserves the same attention as the association method.

Tracker dependence. With AC-MOT, ByteTrack, which starts from a weaker baseline and uses low-score boxes in its second association step, gains more than OATrack, which already weights detections by quality and gains mainly in MOTA, IDF1, IDS, and FP. With SCL, single-stage trackers gain through the

rescue, and two-stage trackers gain when the detector is noisy. The best adaptation therefore depends on the tracker host, and both controllers serve the host without changing it.

Self-limiting by design. In the clean mode SCL keeps the tracker's thresholds and scores, so a well-matched tracker receives its normal input; AOB, which always intervened, lowered the same trackers by 4.15 to 5.89 HOTA points. AC-MOT keeps its state changes rare through its window, holds, and dwell rule, so the detector input stays stable over long stretches of video.

Deployment. Neither controller needs training, labels, or access to the inside of the detector or the tracker. AC-MOT changes three arguments of the detector call and takes about 4.2 ms per frame on a Tesla T4; SCL runs on a CPU in 2.95 ms per frame. Table (4.14) summarizes the two controllers.

Table (4.14) Summary of the two proposed controllers.

Aspect	**AC-MOT**	**SCL**
Acts on	detector operating point: confidence threshold, NMS threshold, input resolution	tracker input: candidates, scores, thresholds, matching tolerance
Reads	image cues of the current frame and tracked boxes of the previous frame	score logits of the last 10 frames, tracks of the previous frame, camera motion
Update	every 10 frames	every frame
Decision	three states from the scene complexity index, with recovery, hysteresis, and dwell	clean or noisy mode from nested Otsu splits and a whole-stream median
Values chosen by	cross-validation on calibration sequences	standard values and design choices, no search
Evaluated on	VisDrone2019-MOT and UAVDT with ByteTrack and OATrack	MOT17, KITTI, and UAVDT with seven published trackers
Main results	VisDrone: HOTA +2.937 and +1.325, MOTA +17.995 and +6.318; UAVDT: MOTA +17.601 and +3.267	halved scores: 0 to 66.17 and 66.72 HOTA; KITTI: MOTA +5.98 and +7.86; UAVDT: MOTA +24.70
Cost	4.26 and 4.20 ms per frame (Tesla T4)	2.95 ms per frame (CPU)

Chapter 5
Conclusions and Future Work

5.1 Conclusion

This thesis studied the operating point of tracking-by-detection systems: the settings of the detector and the thresholds of the tracker that decide which boxes are used, and which are usually fixed for one detector and one type of scene. It proposed two training-free and causal controllers that adapt these settings online, from information that is available while the video is processed, with the detector and the tracker left unchanged. The main findings and contributions are summarized as follows:

1. The detector operating point is a first-order variable of aerial MOT. With the same frozen detector and trackers, moving away from a fixed operating point changed MOTA by up to 17.995 points on VisDrone, mainly by removing false positives and identity-confusing boxes.
2. AC-MOT adapts the confidence threshold, the NMS threshold, and the input resolution of the detector every ten frames from cheap scene cues, through a scene complexity index and a three-state controller with recovery, hysteresis, and dwell rules. Its values were selected by cross-validation on separate calibration sequences and then frozen.
3. On seven VisDrone2019-MOT sequences, AC-MOT improved HOTA by 2.937 points with ByteTrack and 1.325 points with OATrack, and MOTA by 17.995 and 6.318 points, with fewer identity switches; paired bootstrap analysis supports these gains. On UAVDT it improved MOTA by 17.601 and 3.267 points without retuning. It adds about 4.2 ms per frame on a Tesla T4.
4. The score scale of the detector is also part of the tracking system. Halving the scores of a published detector reduced two well-known trackers from 67.70 and 66.43 HOTA to 0, because their fixed thresholds no longer fit the scores.
5. SCL reads the score distribution of recent frames with nested Otsu splits, keeps the tracker's scores and, where they fit, its thresholds on clean scores, and chooses the candidates, rescores them, and sets the thresholds on noisy scores; a continuation rescue gives single-stage trackers the step they lack, and a host contract of four published settings lets one configuration serve different trackers.
6. One frozen SCL configuration restored two trackers from 0 to about 66 HOTA under halved scores, improved both single-stage trackers, including one not seen during design, on every MOT17

sequence, raised MOTA by up to 24.70 points on held-out aerial data, and cut identity switches by 43% to 56% with a noisy detector, while well-matched trackers kept their own accuracy.

7. SCL reached higher HOTA than simple adaptive thresholds, by 2.2 to 8.6 points, and than the published ACT method, and it takes 2.95 ms per frame on an ordinary processor.

Overall, the results show that the operating point of the detector and the score scale that the tracker receives should be treated as parts of the tracking system rather than as fixed settings, and that both can be controlled online, cheaply, and without training.

5.2 Future Work

The proposed controllers can be extended in several directions:

1. Testing both controllers with more detectors, trackers, and aerial and ground-level video datasets.
2. Combining the two controllers in one pipeline, with AC-MOT setting the detector operating point and SCL adapting the tracker input to the resulting scores.
3. Extending the action map of AC-MOT with more operating points and other settings of the detection stage.
4. Measuring both controllers on the low-power processors used on board UAVs, where the cost of the detector input is most important.
5. Extending the host contract of SCL to trackers that use appearance features or learned association, and to methods whose thresholds are internal to the network.

References

[1] P. Zhu, L. Wen, D. Du, X. Bian, H. Fan, Q. Hu, and H. Ling, "Detection and Tracking Meet Drones Challenge," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 7380–7399, 2022.

[2] X. Wu, W. Li, D. Hong, R. Tao, and Q. Du, "Deep Learning for Unmanned Aerial Vehicle-Based Object Detection and Tracking: A survey," *IEEE Geoscience and Remote Sensing Magazine*, vol. 10, no. 1, pp. 91–124, 2022.

[3] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *2012 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3354–3361, IEEE, 2012.

[4] M. Adžemović, "Deep learning-based multi-object tracking: a comprehensive survey from foundations to state-of-the-art," *The Visual Computer*, vol. 42, no. 9, p. 368, 2026.

[5] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *2016 IEEE International Conference on Image Processing (ICIP)*, (Phoenix, AZ, USA), pp. 3464–3468, IEEE, 2016.

[6] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-object Tracking by Associating Every Detection Box," in *Computer Vision – ECCV 2022*, Lecture Notes in Computer Science, (Cham), pp. 1–21, Springer Nature Switzerland, 2022.

[7] D. Du, Y. Qi, H. Yu, Y. Yang, K. Duan, G. Li, W. Zhang, Q. Huang, and Q. Tian, "The Unmanned Aerial Vehicle Benchmark: Object Detection and Tracking," in *Computer Vision – ECCV 2018*, Lecture Notes in Computer Science, (Cham), pp. 375–391, Springer International Publishing, 2018.

[8] G. Cheng, X. Yuan, X. Yao, K. Yan, Q. Zeng, X. Xie, and J. Han, "Towards Large-Scale Small Object Detection: Survey and Benchmarks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, pp. 1–20, 2023.

[9] F. Kuppers, J. Kronenberger, A. Shantia, and A. Haselhoff, "Multivariate Confidence Calibration for Object Detection," in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pp. 1322–1330, IEEE, 2020.

[10] J. Luiten, A. Ošep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixé, and B. Leibe, "HOTA: A Higher Order Metric for Evaluating Multi-object Tracking," *International Journal of Computer Vision*, vol. 129, no. 2, pp. 548–578, 2021.

[11] T.-W. Chin, R. Ding, and D. Marculescu, "AdaScale: Towards Real-time Video Object Detection using Adaptive Scaling," in *Proceedings of Machine Learning and Systems*, vol. 1, pp. 431–441, 2019.

[12] R. Xu, C. lin Zhang, P. Wang, J. Lee, S. Mitra, S. Chaterji, Y. Li, and S. Bagchi, "ApproxDet: Content and Contention-Aware Approximate Object Detection for Mobiles," in *Proceedings of the 18th Conference on Embedded Networked Sensor Systems (SenSys)*, pp. 449–462, ACM, 2020.

[13] J. Jiang, G. Ananthanarayanan, P. Bodik, S. Sen, and I. Stoica, "Chameleon: Scalable Adaptation of Video Analytics," in *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication*, (New York, NY, USA), pp. 253–266, ACM, 2018.

[14] L. V. Ma, M. I. Hussain, J. Park, J. Kim, and M. Jeon, “Adaptive Confidence Threshold for ByteTrack in Multi-Object Tracking,” in *2023 12th International Conference on Control, Automation and Information Sciences (ICCAIS)*, pp. 370–374, IEEE, 2023.

[15] K. Shim, K. Ko, J. Hwang, and C. Kim, “AdapTrack: Adaptive Thresholding-Based Matching for Multi-Object Tracking,” in *2024 IEEE International Conference on Image Processing (ICIP)*, pp. 2222–2228, IEEE, 2024.

[16] D. U. Gwon and W. I. Lee, “Adaptive Threshold Control Scheme Based on Detection Confidence Distribution for Data Association in Multi-Object Tracking,” *Electronics*, vol. 15, no. 17, p. 4047, 2026.

[17] N. Otsu, “A Threshold Selection Method from Gray-Level Histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.

[18] A. G. Ismail, M. S. Mohamed, and T. A. Mahmoud, “Real-Time Object Detection and Tracking: Challenges, Innovations and Future Directions,” in *International Conference ICMISI 2026*, 2026.

[19] S. Li, H. Ren, X. Xie, and Y. Cao, “A Review of Multi-Object Tracking in Recent Times,” *IET Computer Vision*, vol. 19, no. 1, 2025.

[20] N. Dalal and B. Triggs, “Histograms of Oriented Gradients for Human Detection,” in *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR’05)*, pp. 886–893, IEEE, 2005.

[21] D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.

[22] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” *Communications of the ACM*, vol. 60, no. 6, pp. 84–90, 2017.

[23] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, “ImageNet Large Scale Visual Recognition Challenge,” *International Journal of Computer Vision*, vol. 115, no. 3, pp. 211–252, 2015.

[24] R. Girshick, J. Donahue, T. Darrell, and J. Malik, “Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation,” in *2014 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 580–587, IEEE, 2014.

[25] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, 2017.

[26] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788, IEEE, 2016.

[27] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, “SSD: Single Shot MultiBox Detector,” in *Computer Vision – ECCV 2016*, Lecture Notes in Computer Science, pp. 21–37, Springer International Publishing, 2016.

[28] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, IEEE, 2016.

[29] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention Is All You Need.” arXiv preprint arXiv:1706.03762, 2017.

[30] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale." arXiv preprint arXiv:2010.11929, 2020.

[31] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-End Object Detection with Transformers," in *Computer Vision – ECCV 2020*, Lecture Notes in Computer Science, pp. 213–229, Springer International Publishing, 2020.

[32] P. Sun, J. Cao, Y. Jiang, R. Zhang, E. Xie, Z. Yuan, C. Wang, and P. Luo, "TransTrack: Multiple Object Tracking with Transformer." arXiv preprint arXiv:2012.15460, 2020.

[33] T. Meinhardt, A. Kirillov, L. Leal-Taixe, and C. Feichtenhofer, "TrackFormer: Multi-Object Tracking with Transformers," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 8834–8844, IEEE, 2022.

[34] F. Zeng, B. Dong, Y. Zhang, T. Wang, X. Zhang, and Y. Wei, "MOTR: End-to-End Multiple-Object Tracking with Transformer," in *Computer Vision – ECCV 2022*, Lecture Notes in Computer Science, pp. 659–675, Springer Nature Switzerland, 2022.

[35] M. Bakirci, "Performance evaluation of low-power and lightweight object detectors for real-time monitoring in resource-constrained drone systems," *Engineering Applications of Artificial Intelligence*, vol. 159, p. 111775, 2025.

[36] N. Miller and J. Rogers, "Simultaneous Tracking of Multiple Ground Targets from a Multirotor Unmanned Aerial Vehicle," *Journal of Aerospace Information Systems*, vol. 12, no. 3, pp. 345–364, 2015.

[37] A. Desai and D.-J. Lee, "Efficient Feature Descriptor for Unmanned Aerial Vehicle Ground Moving Object Tracking," *Journal of Aerospace Information Systems*, vol. 14, no. 6, pp. 345–350, 2017.

[38] Y.-R. Lu and D. Sun, "Development of Real-Time Unmanned Aerial Vehicle Urban Object Detection System with Federated Learning," *Journal of Aerospace Information Systems*, vol. 21, no. 7, pp. 547–553, 2024.

[39] A. J. Choi, J. Park, and J.-H. Han, "Automated Aerial Docking System Using Onboard Vision-Based Deep Learning," *Journal of Aerospace Information Systems*, vol. 19, no. 6, pp. 421–436, 2022.

[40] C. Parsons, Z. Paulson, S. Nykl, W. Dallman, B. G. Woolley, and J. Pecarina, "Analysis of Simulated Imagery for Real-Time Vision-Based Automated Aerial Refueling," *Journal of Aerospace Information Systems*, vol. 16, no. 3, pp. 77–93, 2019.

[41] Y. Chae, H. Kim, and K.-J. Yoon, "Towards Robust 3D Object Detection with LiDAR and 4D Radar Fusion in Various Weather Conditions," in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 15162–15172, IEEE, 2024.

[42] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.

[43] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.

[44] N. Wojke, A. Bewley, and D. Paulus, “Simple online and realtime tracking with a deep association metric,” in *2017 IEEE International Conference on Image Processing (ICIP)*, pp. 3645–3649, IEEE, 2017.

[45] Z. Wang, L. Zheng, Y. Liu, Y. Li, and S. Wang, “Towards Real-Time Multi-Object Tracking,” in *Computer Vision – ECCV 2020*, Lecture Notes in Computer Science, pp. 107–122, Springer International Publishing, 2020.

[46] Y. Zhang, C. Wang, X. Wang, W. Zeng, and W. Liu, “FairMOT: On the Fairness of Detection and Re-identification in Multiple Object Tracking,” *International Journal of Computer Vision*, vol. 129, no. 11, pp. 3069–3087, 2021.

[47] X. Zhou, V. Koltun, and P. Krähenbühl, “Tracking Objects as Points,” in *Computer Vision – ECCV 2020*, Lecture Notes in Computer Science, pp. 474–490, Springer International Publishing, 2020.

[48] R. Gao, J. Qi, and L. Wang, “Multiple Object Tracking as ID Prediction,” in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 27883–27893, IEEE, 2025.

[49] K. He, G. Gkioxari, P. Dollar, and R. Girshick, “Mask R-CNN,” in *2017 IEEE International Conference on Computer Vision (ICCV)*, pp. 2980–2988, IEEE, 2017.

[50] J. Redmon and A. Farhadi, “YOLOv3: An Incremental Improvement.” arXiv preprint arXiv:1804.02767, 2018.

[51] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection.” arXiv preprint arXiv:2004.10934, 2020.

[52] Z. Ge, S. Liu, F. Wang, Z. Li, and J. Sun, “YOLOX: Exceeding YOLO Series in 2021.” arXiv preprint arXiv:2107.08430, 2021.

[53] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, “YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information,” in *Computer Vision – ECCV 2024*, Lecture Notes in Computer Science, pp. 1–21, 2024.

[54] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8.” Software package, Ver. 8.0.0, Ultralytics, https://github.com/ultralytics/ultralytics, 2023.

[55] G. Jocher and J. Qiu, “Ultralytics YOLO11.” Software package, Ver. 11.0.0, Ultralytics, https://github.com/ultralytics/ultralytics, 2024.

[56] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, “Focal Loss for Dense Object Detection,” in *2017 IEEE International Conference on Computer Vision (ICCV)*, pp. 2999–3007, IEEE, 2017.

[57] M. Tan, R. Pang, and Q. V. Le, “EfficientDet: Scalable and Efficient Object Detection,” in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10778–10787, IEEE, 2020.

[58] T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie, “Feature Pyramid Networks for Object Detection,” in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 936–944, IEEE, 2017.

[59] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, “DETRs Beat YOLOs on Real-time Object Detection,” in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 16965–16974, IEEE, 2024.

[60] I. Robinson, P. Robicheaux, M. Popov, D. Ramanan, and N. Peri, “RF-DETR: Neural Architecture Search for Real-Time Detection Transformers.” arXiv preprint arXiv:2511.09554, 2025.

[61] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On Calibration of Modern Neural Networks.” arXiv preprint arXiv:1706.04599, 2017.

[62] N. Bodla, B. Singh, R. Chellappa, and L. S. Davis, “Soft-NMS — Improving Object Detection with One Line of Code,” in *2017 IEEE International Conference on Computer Vision (ICCV)*, pp. 5562–5570, IEEE, 2017.

[63] D. Wang, E. Shelhamer, S. Liu, B. Olshausen, and T. Darrell, “Tent: Fully Test-time Adaptation by Entropy Minimization.” arXiv preprint arXiv:2006.10726, 2020.

[64] E. Bochinski, V. Eiselein, and T. Sikora, “High-Speed tracking-by-detection without using image information,” in *2017 14th IEEE International Conference on Advanced Video and Signal Based Surveillance (AVSS)*, pp. 1–6, IEEE, 2017.

[65] J. Cao, J. Pang, X. Weng, R. Khirodkar, and K. Kitani, “Observation-Centric SORT: Rethinking SORT for Robust Multi-Object Tracking,” in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9686–9696, IEEE, 2023.

[66] F. Yang, S. Odashima, S. Masui, and S. Jiang, “Hard to Track Objects with Irregular Motions and Similar Appearances? Make It Easier by Buffering the Matching Space,” in *2023 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pp. 4788–4797, IEEE, 2023.

[67] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky, “BoT-SORT: Robust Associations Multi-Pedestrian Tracking.” arXiv preprint arXiv:2206.14651, 2022.

[68] G. Evangelidis and E. Psarakis, “Parametric Image Alignment Using Enhanced Correlation Coefficient Maximization,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 10, pp. 1858–1865, 2008.

[69] Y. Du, Z. Zhao, Y. Song, Y. Zhao, F. Su, T. Gong, and H. Meng, “StrongSORT: Make DeepSORT Great Again,” *IEEE Transactions on Multimedia*, vol. 25, pp. 8725–8737, 2023.

[70] G. Maggiolino, A. Ahmad, J. Cao, and K. Kitani, “Deep OC-Sort: Multi-Pedestrian Tracking by Adaptive Re-Identification,” in *2023 IEEE International Conference on Image Processing (ICIP)*, pp. 3025–3029, IEEE, 2023.

[71] Y.-H. Wang, J.-W. Hsieh, P.-Y. Chen, M.-C. Chang, H.-H. So, and X. Li, “SMILEtrack: SiMIlarity LEarning for Occlusion-Aware Multiple Object Tracking,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 6, pp. 5740–5748, 2024.

[72] Y. Li, Y. Liu, C. Zhou, D. Xu, and W. Tao, “A lightweight scheme of deep appearance extraction for robust online multi-object tracking,” *The Visual Computer*, vol. 40, no. 3, pp. 2049–2065, 2024.

[73] Y. Bao, Y. Yu, Y. Qi, and Z. Wang, “Multiple object tracking with adaptive multi-features fusion and improved learnable graph matching,” *The Visual Computer*, vol. 40, no. 4, pp. 2279–2292, 2024.

[74] Y. Zhang, Y. Liang, J. Wang, H. Zhu, and Z. Wang, “Enhanced multi-object tracking via embedded graph matching and differentiable Sinkhorn assignment: addressing challenges in occlusion and varying object appearances,” *The Visual Computer*, vol. 41, no. 8, pp. 6067–6085, 2025.

[75] Y. Ding, Y. Wang, L. Wang, B. Zhang, J. Li, L. Guo, Z. Yang, and W. Liang, “Enhancing multi-object tracking efficiency through result-guided feature extraction and query filtering,” *The Visual Computer*, vol. 41, no. 11, pp. 8153–8167, 2025.

[76] Z. Liang, G. Xiao, J. Hu, J. Wang, and C. Ding, “MotionTrack: rethinking the motion cue for multiple object tracking in USV videos,” *The Visual Computer*, vol. 40, no. 4, pp. 2761–2773, 2024.

[77] Z. Sun, Z. Lv, M. Xiong, W. Hu, G. Wei, J. Zhou, T. He, and Q. Meng, “Online visual multi-object tracking via blind super-resolution and clustering learning,” *The Visual Computer*, vol. 42, no. 4, p. 189, 2026.

[78] M. Yang, G. Han, B. Yan, W. Zhang, J. Qi, H. Lu, and D. Wang, “Hybrid-SORT: Weak Cues Matter for Online Multi-Object Tracking,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 7, pp. 6504–6512, 2024.

[79] Z. Liu, X. Wang, C. Wang, W. Liu, and X. Bai, “SparseTrack: Multi-Object Tracking by Performing Scene Decomposition Based on Pseudo-Depth,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 35, no. 5, pp. 4870–4882, 2025.

[80] V. D. Stanojevic and B. T. Todorovic, “BoostTrack: boosting the similarity measure and detection confidence for improved multiple object tracking,” *Machine Vision and Applications*, vol. 35, no. 3, 2024.

[81] Y. Wang, D. Zhang, R. Li, Z. Zheng, and M. Li, “PD-SORT: Occlusion-Robust Multi-Object Tracking Using Pseudo-Depth Cues,” *IEEE Transactions on Consumer Electronics*, vol. 71, no. 1, pp. 165–177, 2025.

[82] K. Liu, S. Jin, Z. Fu, Z. Chen, R. Jiang, and J. Ye, “Uncertainty-aware Unsupervised Multi-Object Tracking,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 9996–10005, 2023.

[83] L. Qi, L. Wang, and S. Jiang, “OATrack: A Quality-Gated Progressive Association Framework with a YOLO Detection Cache for UAV Small-Object Multi-Object Tracking,” *Sensors*, vol. 26, no. 16, p. 5222, 2026.

[84] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: Common Objects in Context,” in *Computer Vision – ECCV 2014*, Lecture Notes in Computer Science, (Cham), pp. 740–755, Springer International Publishing, 2014.

[85] A. Milan, L. Leal-Taixe, I. Reid, S. Roth, and K. Schindler, “MOT16: A Benchmark for Multi-Object Tracking.” arXiv preprint arXiv:1603.00831, 2016.

[86] P. Dendorfer, A. Ošep, A. Milan, K. Schindler, D. Cremers, I. Reid, S. Roth, and L. Leal-Taixé, “MOTChallenge: A Benchmark for Single-Camera Multiple Target Tracking,” *International Journal of Computer Vision*, vol. 129, no. 4, pp. 845–881, 2021.

[87] P. Dendorfer, H. Rezatofighi, A. Milan, J. Shi, D. Cremers, I. Reid, S. Roth, K. Schindler, and L. Leal-Taixé, “MOT20: A benchmark for multi object tracking in crowded scenes.” arXiv preprint arXiv:2003.09003, 2020.

[88] P. Sun, J. Cao, Y. Jiang, Z. Yuan, S. Bai, K. Kitani, and P. Luo, “DanceTrack: Multi-Object Tracking in Uniform Appearance and Diverse Motion,” in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 20961–20970, IEEE, 2022.

[89] Y. Han, G. Huang, S. Song, L. Yang, H. Wang, and Y. Wang, “Dynamic Neural Networks: A Survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 7436–7456, 2022.

[90] L. Yang, Y. Han, X. Chen, S. Song, J. Dai, and G. Huang, “Resolution Adaptive Networks for Efficient Inference,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2369–2378, 2020.

[91] H. Jung, S. Kang, T. Kim, and H. Kim, “ConfTrack: Kalman Filter-based Multi-Person Tracking by Utilizing Confidence Score of Detection Box,” in *2024 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pp. 6569–6578, 2024.

[92] E. Solano-Carrillo, F. Sattler, A. Alex, A. Klein, B. P. Costa, A. B. Rodriguez, and J. Stoppe, “UTrack: Multi-object Tracking with Uncertain Detections,” in *Computer Vision – ECCV 2024 Workshops*, Lecture Notes in Computer Science, (Cham), pp. 219–236, Springer Nature Switzerland, 2025.

[93] S. Kim, S. Lee, and M. Cho, “PlugTrack: Multi-Perceptive Motion Analysis for Adaptive Fusion in Multi-Object Tracking,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, pp. 5709–5716, 2026.

[94] C. Kanellakis and G. Nikolakopoulos, “Survey on Computer Vision for UAVs: Current Developments and Trends,” *Journal of Intelligent & Robotic Systems*, vol. 87, no. 1, pp. 141–168, 2017.

[95] S. Liu, X. Li, H. Lu, and Y. He, “Multi-Object Tracking Meets Moving UAV,” in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 8866–8875, IEEE, 2022.

[96] S. Kim, I. Petrunin, and H.-S. Shin, “AFJPDA: A Multiclass Multi-Object Tracking with Appearance Feature-Aided Joint Probabilistic Data Association,” *Journal of Aerospace Information Systems*, vol. 21, no. 4, pp. 294–304, 2024.

[97] G. Huang, X. Zhang, R. Zhao, W. Li, B. Liang, and J. Xie, “Efficient Small-Object Detection in Airport Surface Based on Maintain Feature High Resolution,” *Journal of Aerospace Information Systems*, vol. 19, no. 4, pp. 305–316, 2022.

[98] M. Fathy, M. K. Elhadad, and M. A. Elshafey, “TOReN: Tiny Object Recognition Network in Remote Sensing Imagery,” *Journal of Aerospace Information Systems*, vol. 23, no. 8, pp. 784–803, 2026.

[99] P. Dai, L. Chen, Y. Lian, and L. Xie, “LSO-YOLO: a lightweight real-time object detection network for UAV small-object detection,” *Journal of Real-Time Image Processing*, vol. 23, no. 5, p. 176, 2026.

[100] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A Next-generation Hyperparameter Optimization Framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, (Anchorage, AK, USA), pp. 2623–2631, ACM, 2019.

[101] K. Bernardin and R. Stiefelhagen, “Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics,” *EURASIP Journal on Image and Video Processing*, vol. 2008, pp. 1–10, 2008.

[102] E. Ristani, F. Solera, R. Zou, R. Cucchiara, and C. Tomasi, “Performance Measures and a Data Set for Multi-target, Multi-camera Tracking,” in *Computer Vision – ECCV 2016 Workshops*, Lecture Notes in Computer Science, pp. 17–35, Springer International Publishing, 2016.

[103] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York: Springer US, 1993.

[104] J. Pech-Pacheco, G. Cristobal, J. Chamorro-Martinez, and J. Fernandez-Valdivia, “Diatom autofocusing in brightfield microscopy: a comparative study,” in *Proceedings 15th International Conference on Pattern Recognition. ICPR-2000*, pp. 314–317, IEEE Comput. Soc, 2000.

[105] B. Reddy and B. Chatterji, “An FFT-based technique for translation, rotation, and scale-invariant image registration,” *IEEE Transactions on Image Processing*, vol. 5, no. 8, pp. 1266–1271, 1996.

Publications

[1] A. G. Ismail, M. S. Mohamed, and T. A. Mahmoud, “Real-Time Object Detection and Tracking: Challenges, Innovations and Future Directions,” in *2026 IEEE 3rd International Conference on Machine Intelligence and Smart Innovation (ICMISI)*, Alexandria, Egypt, 2026, pp. 119–125, doi: 10.1109/ICMISI69868.2026.11583994.

[2] A. G. Ismail, M. S. Mohamed, and T. A. Mahmoud, “Optimization-Guided Adaptive Detector Operating-Point Control for Multi-Object Aerial Tracking,” *Journal of Aerospace Information Systems*, submitted (under review).

[3] A. G. Ismail, M. S. Mohamed, and T. A. Mahmoud, “Training-Free Self-Calibrating Control Layer for Heterogeneous Multi-Object Trackers,” to be submitted to *The Visual Computer*.

ملخص الرسالة

يحدد تتبع الأجسام المتعددة كل جسم مهم في الفيديو ويمنحه هوية ثابتة عبر الزمن. وفي أسلوب التتبع بالاعتماد على الكشف يعطي الكاشف في كل إطار صناديق مصحوبة بدرجات ثقة، ثم يربطها المتتبع في مسارات. ويعمل كلا الجزأين بإعدادات ثابتة تسمى نقطة التشغيل، مع أن المشهد يتغير داخل الفيديو الواحد، ولا سيما في فيديو الطائرات دون طيار، ومع أن الكاشف يُستبدل أو يُعاد تدريبه دون إعادة ضبط المتتبع. وتقترح هذه الرسالة متحكمَين لا يحتاجان إلى تدريب ولا يستخدمان إلا المعلومات المتاحة حتى الإطار الحالي، يكيّفان نقاط التشغيل أثناء التشغيل دون بيانات معنونة ودون تعديل الكاشف أو المتتبع.

المتحكم الأول (AC-MOT) يضبط عتبة الثقة وعتبة إخماد الصناديق المتداخلة ودقة الإدخال للكاشف؛ ففي كل عشرة إطارات يكوّن مؤشراً لتعقيد المشهد من إشارات منخفضة التكلفة، ويختار إحدى ثلاث نقاط تشغيل عبر متحكم ثلاثي الحالات، وقد اختيرت قيمه بالتحقق المتقاطع على تسلسلات معايرة منفصلة ثم جُمّدت. وعلى سبعة تسلسلات من VisDrone2019-MOT رفع دقة HOTA بمقدار 2.937 نقطة مع ByteTrack و 1.325 نقطة مع OATrack، ورفع دقة MOTA بمقدار 17.995 و6.318 نقطة، وعلى قاعدة UAVDT دون إعادة ضبط رفع MOTA بمقدار 17.601 و3.267 نقطة، بتكلفة نحو 4.2 مللي ثانية لكل إطار.

أما المتحكم الثاني فطبقة تحكم ذاتية المعايرة (SCL) تقع بين كاشف ومتتبع دون تغيير أي منهما؛ إذ تقسم لوغاريتمات الأرجحية لدرجات الإطارات الأخيرة مرتين بطريقة أوتسو، فتحتفظ بدرجات المتتبع وعتباته عندما يكون توزيع الدرجات نظيفاً، وتختار في غير ذلك المرشحين وتعطيهم درجات جديدة وتضبط العتبات. وقد اختُبر إعداد واحد مجمِّد على سبعة متتبعات منشورة وأربعة مصادر للكشوف وثلاث قواعد بيانات؛ فأعاد متتبعَين من دقة صفرية إلى نحو 66 عند تنصيف الدرجات، وحسّن المتتبعَين أحاديي المرحلة على كل تسلسلات MOT17، ورفع MOTA بما يصل إلى 24.70 نقطة مع كاشف مشوش على بيانات جوية محجوزة، وحقق دقة أعلى من العتبات التكيفية البسيطة ومن طريقة ACT المنشورة، بتكلفة 2.95 مللي ثانية لكل إطار على معالج عادي.

ويُظهر المتحكمان معاً أن نقطة تشغيل الكاشف ومقياس الدرجات الذي يتلقاه المتتبع جزءان من منظومة التتبع يمكن التحكم فيهما أثناء التشغيل بتكلفة منخفضة ودون تدريب.

الكلية الفنية العسكرية

قسم هندسة الحواسب والذكاء الاصطناعى

التحكم التكيفي دون تدريب في نقاط تشغيل الكاشف والمتتبع لتتبع الأجسام المتعددة

رسالة مقدمة من

أحمد جودة إسماعيل

الكلية الفنية العسكرية

للحصول على درجة الماجستير فى هندسة الحاسبات

تحت إشراف

أ.م.د./ محمد س. محمد

الكلية الفنية العسكرية

أ.د./ طارق أحمد محمود

جامعة مصر للمعلوماتية

القاهرة 2026